

Algo : Gauss Jordan - version distribuée

I - INTRODUCTION

Def : Une machine peut - être

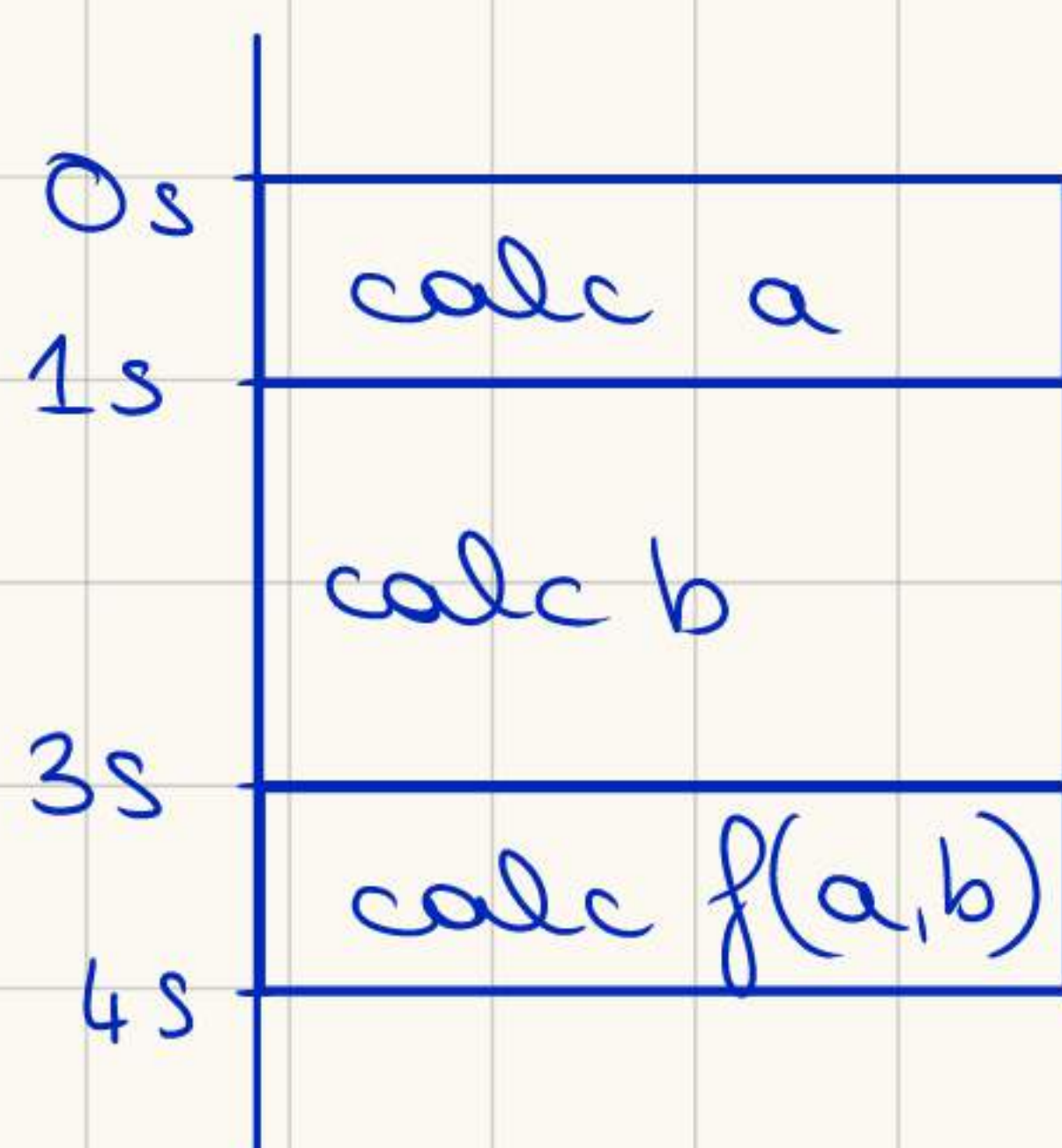
- * séquentielle
- * parallèle (qd on a plus cœurs)
- * distribuée (batteries d'ordi reliés en réx)
 - ↳ 3 pbs :
 - * temps de comm° = lent
 - * mém. pas partagée
 - * passage de msg (MPI) on va le pr en C

↗ = entités de calcul (= cœurs)

Ex : Imaginons qu'on a 2 mach (qui share pas mém)
 Objectif : calculer $f(a, b)$ mais calculer a (1 sec)
 et calc b (2 sec) c'est "long"

mais calc $f(a, b)$ une fois qu'on a a et b c'est rapide

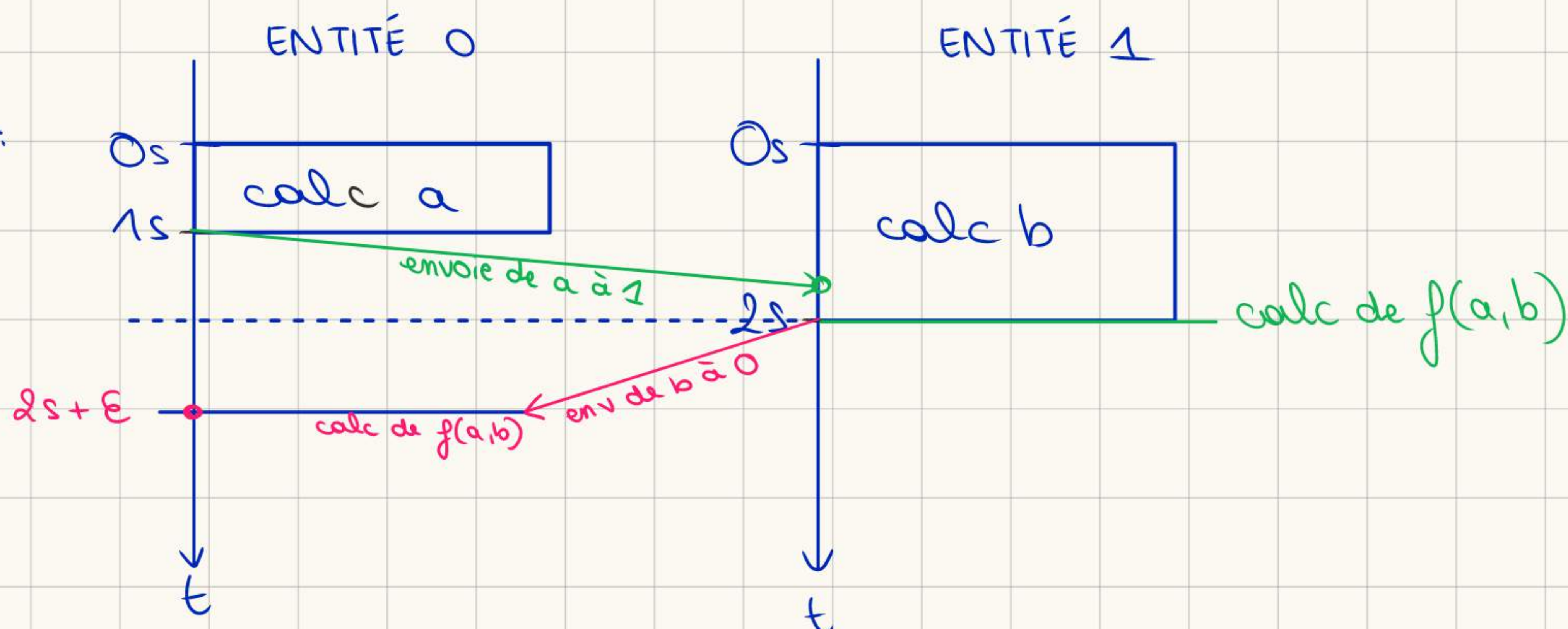
version séquentielle :



ms ft in mach de l'un va env msg à l'autre

version distribuée :

opt° 1
opt° 2



mém shared $\Rightarrow$ comm° peu coûteuse
 mém dist $\Rightarrow$ comm° coûteuse via msg

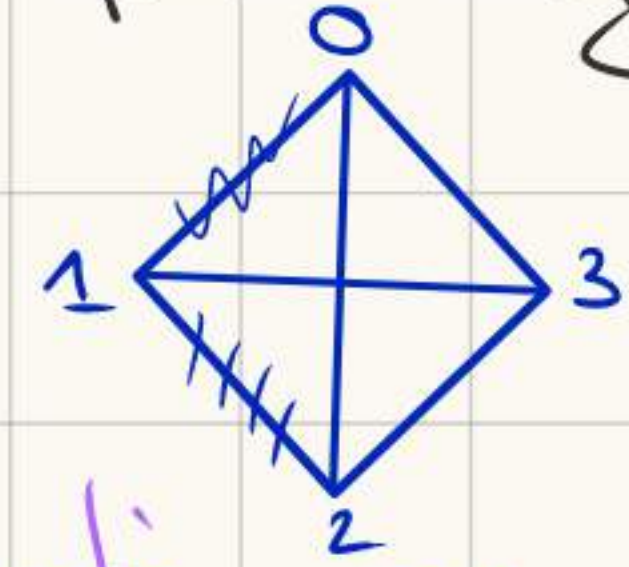
I 1) Architectures

opt 1 prend 3 sec opt 2 prend 3+E sec

3 ent put f^r m chose via 2 SD ≠

① I ↔ II ② I → II → III

I 2) Communic^o par pssg de msg

- * Il y a 1 seul même prog pour toutes les entités
 → y a tjrs qlq entités qui marchent pas donc id qui mène parfois à n → nous en consid^r tout ok
 - * chq entité possède un identifiant : un nb i entre 0 et $n-1$ avec n le nb total d'entités de calcul
 - * toutes les entités de calc ont un lien direct avec toutes les autres. le graphe d'un syst dist ds l'interface de pssg par msg (MPI) est le graphe complet (on le suppose pssg par tjrs vrai)
- 
- * une enti ne coco que les mach auxq^l elle est liée
 → de si une co se perd 0-1 ft passer par une autre : 0-3-1
 - * lorsqu' une entité att un msg elle ne pt pas faire de calcul avant que ce msg soit reçu
 → receive est bloquante
 (la plus des bugs = le prog se blo car msg attendu jms sent)
- Au Rd MPI c' est juste d' instruct^o:
- * Il y a une instruct^o d' envoie et une instruct^o de recept^o

Ex: Reprenons l'ex du calc de $f(a, b)$
 On voudrait :

ENTITÉ 0
 Calculer a
 Envoyer a à 1

ENTITÉ 1
 Calculer b
 Recevoir a de 0
 Calculer $f(a, b)$

Version prog sur une m machine:

Si $i = 0$ alors
 Calculer a
 Envoyer a à 1

Si $i = 1$ alors
 Calculer b
 Recevoir a de 0
 Calculer $f(a, b)$
 Fin m

Exo 1: $\left(\begin{array}{l} i = \text{l'identifiant de l'entité de calcul} \\ \text{et } m = \text{un entier} \end{array} \right)$

Envoyer i à $(i+1) \% 2$
 Recevoir m de $(i+1) \% 2$
 Afficher m
 Tracer ce prog :

passer de pseudo-code
 à sa trace

Ent 0	Ent 1
Env 0 à 1 (id à qui)	Env 1 à 0
Rec m de 1 [m=1]	Rec m de 0 [m=0]
Afficher 1	Aff 0

Exo 2: Poser $k \leftarrow 0, n \leftarrow 0, e \leftarrow 0$

Tant que $k < 4$ faire

donc ça ne
 pt qu'être 0 ou 1

Si $k \% 2 = i$ alors
reste mod 2

Envoyer k à $(i+1) \% 2$
 $\div k$ par module de 2 $\rightarrow 0$ ou 1
 Poser $e \leftarrow e + 1$

Sinon

Recevoir k de $(i+1) \% 2$
 Poser $n \leftarrow n + 1$

Fin si

Poser $k \leftarrow k + 1$

Fin tant que
 Afficher e, n

passer de pseudo-code
 à sa trace

Correction:

ENT 0	ENT 1
Poser k, e, r à 0	Poser k, e, r à 0
$0 \% 2 = \text{ENT} 0$ 1 Env 0 à 1	$0 \% 2 \neq \text{ENT} 1$ 3 Rec $k \Rightarrow ? 0$ de 0
Poser e à 1 et k à 1	Poser r à 1 et k à ? $0 + 1 = 1$
$1 \% 2 \neq \text{ENT} 0$ 2 Rec $k = ? 1$ de 1	$1 \% 2 = \text{ENT} 1$ 4 Env 1 à 0
Poser r à 1 et k à ? $1 + 1 = 2$	Poser e à 1 et k à 2
$2 \% 2 = \text{ENT} 0$ 6 Env 2 à 1	$2 \% 2 \neq \text{ENT} 1$ 5 Rec $k \Rightarrow ? 2$ de 0
Poser e à 2 et k à 3	Poser r à 2 et k à ? $2 + 1 = 3$
$3 \% 2 = 1 \neq \text{ENT} 0$ 7 Rec $k = ? 3$ de 1	$3 \% 2 = 1 = \text{ENT} 1$ 8 Env 3 à 0
Poser r à 2 et k à ? $3 + 1 = 4$	Poser e à 2 et k à 4
10 Afficher $e = 2$ et $r = 2$	9 Afficher $e = 2$ et $r = 2$

MPI:

Compiler : mpicc -Wall -o executable source.c
Lancer : mpirun -np 6 (--oversubscribe) executable
↳ move 6 à la fin
↳ 2 --- 6

```
#include <stdio>
#include <stdlib>
#include "mpi.h" → changt de librairie MPI
```

```
int main()
```

```
{
    int i, n, k, r, e;
    MPI_Init(NULL, NULL);
    MPI_Comm_size(MPI_COMM_WORLD, &n); % affecter la val à n
    MPI_Comm_rank(MPI_COMM_WORLD, &i); % affecter la val à i
    k = 0;
    r = 0;
    e = 0;
    while (k < 4)
    {
        if (k % 2 == i)
        {
            MPI_Send(&k, 1, MPI_INT, (i+1)%2, 0, MPI_COMM_WORLD);
            e++;
        }
        else
        {
            MPI_Status status;
            MPI_Recv(&k, 1, MPI_INT, (i+1)%2, 0, MPI_COMM_WORLD, &status);
            r++;
        }
    }
}
```

→ = communicateur
= env des ent de calc
passer en arg
↑ adresse
entité
↑
identifiant

on pt envoy donnée vs info (ex données = résultat vs info = accusé de receipt)
2 tues de m type
pt avoir sens ≠

nb de données
nb d'ent puis le type
balise = deux pr msg
balise de
type de var.
pas obligé : si on s'attend à un msg, qu'on vt juste le 1er qui arrive.
coco pas taille du msg
MPI_ANY_TAG
MPI_ANY_SOURCE

1er des else = recevoir.

```

    k++;
}
printf("%d %d\n", e, r);
MPI_Finalize();
return 0;
}

```

il va falloir que 0 send à 1 et 2 "tu es mon vois"
 ms pr que 1 receive ft coco nb de msg ↴

de chq entité va send un msg à tout le monde
 en précisant 0
 1

(Dijkstra distribué)

Exo 3: Soient n entités de calcul $i = 0$ à $n-1$

L'entité i est voisine de $(i+1) \% n$ et $(i-1+n) \% n$

(a) Représenter le graphe de ce syst dist

(b) Écrire le pseudo-code qui fait:

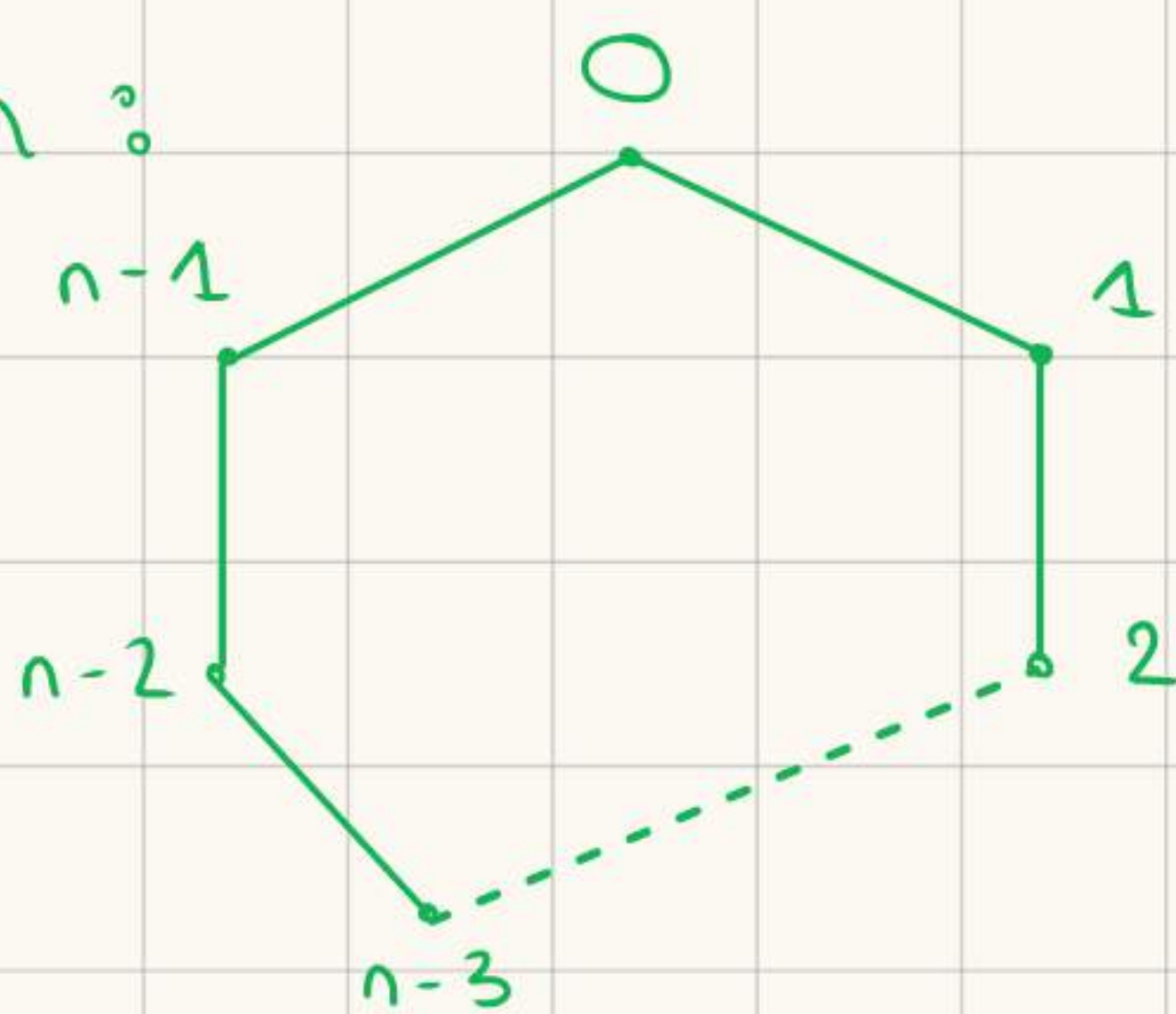
- 0 envoie 0 à 1
- $i \neq 0$ reçoivent k de $i-1$ et envoient $k+i$ à $(i+1) \% n$
- 0 reçoit k de $n-1$ et l'affiche

obligé de mettre $\% n$

→ ici $i-1 = (i-1+n) \% n$

correction:

(a)



les pointillés indiquent les entités
 de 3 à $n-4$.

Ce graphe est une boucle.



(b)

Si $i = 0$

Envoyer 0 à 1.

Recevoir k de $n-1$. ←

Afficher k

Sinon

Recevoir k de $i-1$

Envoyer $k+i$ à $(i+1) \% n$

fin si

EO sera bloqué

ici jusqu'à fin

ms c les E font

tous ce prog, tous les

autres E que EO feront

leur truc (sinon)

et de $n-1$ enverra à EO

et EO va recevoir son k et cont

TP: (95)

- 0 envoie m à 1
- 0 reçoit k de 1
- 1 reçoit k de 0
- 1 envoie k à 1

pseudo-code:

```
if (id = 0)
    Env m à 1
    Rec q de 1
else
    Rec k de 0
    Env q à 0
```

(97) voir (★)
n proc

- 0 env k à 1
 - 0 rec m de n-1
- printf

- graphes
- cplx dist
- diffus^o → algo CAST

TP

I3) Rappels de théorie des graphes

G: un graphe { dont les sommets sont les entités de calcul
 { dont les arêtes sont les communic^o directes

n: nb d'entités (= nb de sommets)

m: nb d'arêtes de G.

q^o: comment calculer des dist dans G?

Def: chemin dans G est une suite $v_1, \dots, v_k$ de sommets de G telle que v_i et v_{i+1} sont reliés par une arête de G ($\forall i$ tq $1 \leq i \leq k-1$) la lg d'un tel chem = k

1

0, 2 est un chemin entre 0 et 2

0, 1, 2

2

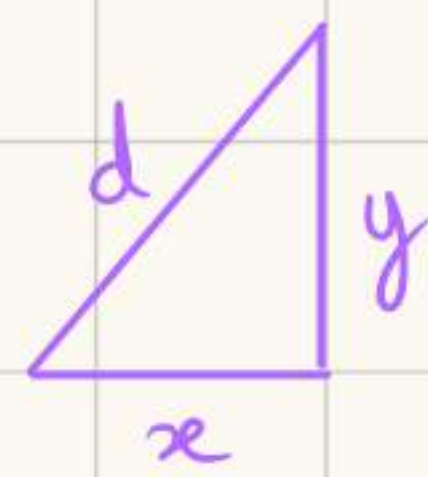
0, 1, 0, 1, 2

0, 2 est un $\oplus$ court chemin entre 0 et 2 dans G

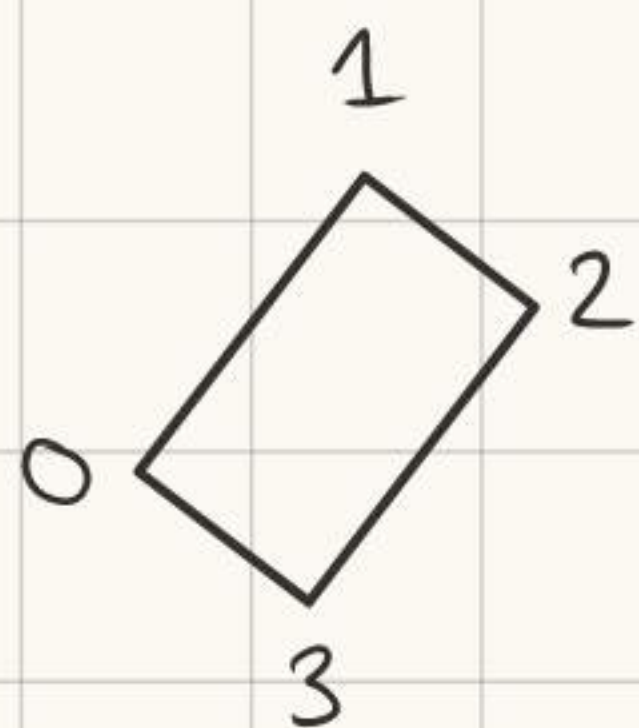
$\rightarrow$ lg chem = nb d'arêtes

graphe connexe $\Leftarrow \forall$ paire de sommets u et v de G , $\exists$ chem entre u et v

dist euclid



$$d = \sqrt{x^2 + y^2}$$



ici il y a 2 $\oplus$ courts chem entre 0 et 2 (de lg 2)

Def: la dist entre 2 sommets de G est la lg (en nb d'arêtes) d'un $\oplus$ court chemin entre 2 sommets.

complexité = au pire

Def: le diam de G est la $\oplus$ gde dist entre 2 de ses sommets

$\hookrightarrow$ c'est un max de min.

$$G \text{ connexe} \Rightarrow \text{diam}(G) = \max_{u,v \in V} d(u,v)$$

Def: le degré du sommet $v \in V$ dans G est le nb de voisins de v dans G ou encore le nb d'arête de la forme $\{v, w\}$. On le note $\deg_G(v)$.

= nb de sommets avec qu'il share 1 arête

Proprio: $2m = \sum_{u \in V} \deg_G(u)$

incidences entre sommet et arête

perm de calc cplx^T en nb de msg

preuve: puisqu qu'une arête est incidente à 2 sommets, # incidence ar-som = 2 # sommets

ray

Def: L'excentricité d'un sommet v de G est la plus gde dist poss entre v et un (autre) sommet de G .

$\hat{c}$ diam
ms un des 2 pts est fixé, dist max entre 1 pt fixe et 1 pt quelc

diam
dist max entre 2 pts quelc

$$e_G(v) = \max_{w \in V} d(v, w)$$

max $w \in V$

$\neq$

max $u, v \in V$

diam est la $\oplus$ gde de ttes ses excentricités.

$\rightarrow$ la $\oplus$ petite de ttes ses excentricités

Def: l'excentricité du graphe G est l'excentricité la plus petite parmi les sommets de G :

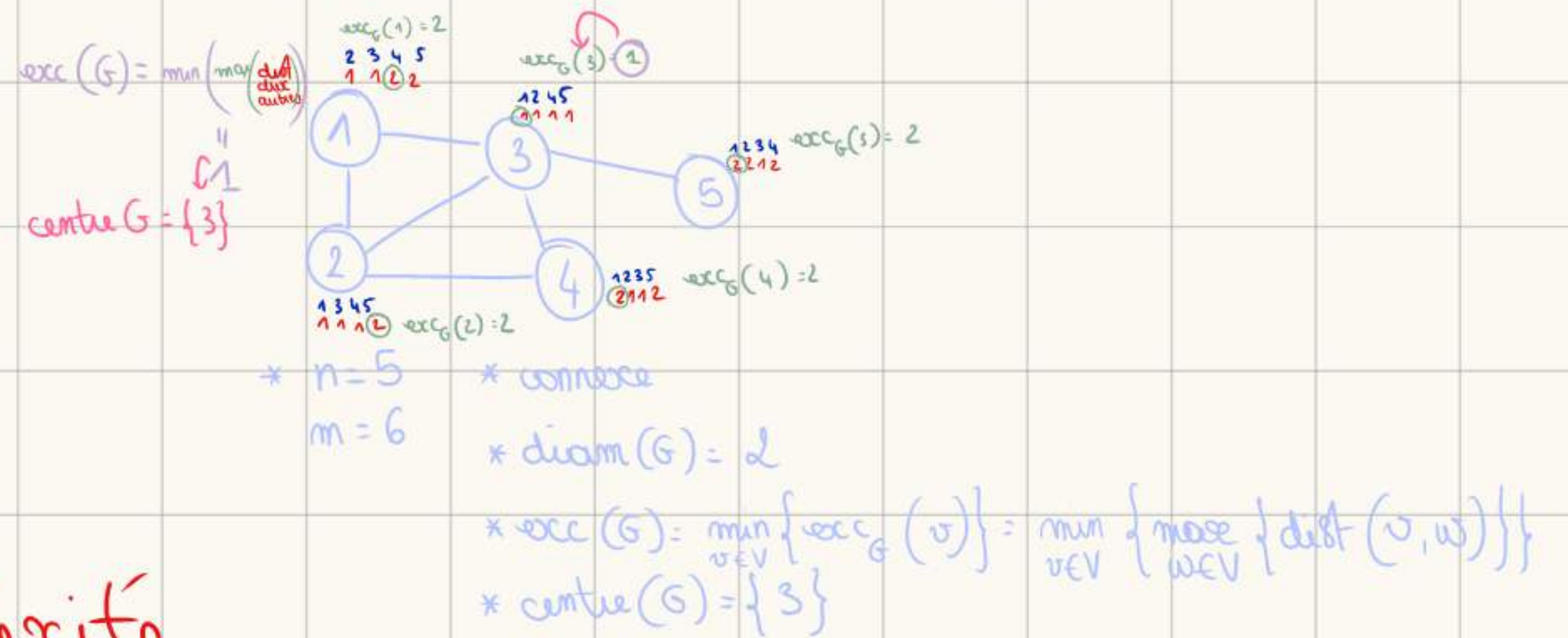
$$ecc(G) = \min_{v \in V} \{ecc_G(v)\} \rightarrow \text{min de max}$$

Def: le centre de G est l'ens des sommets v de G tels que $ecc_G(v) = ecc(G)$

excentricité en ce sommet = min des excentricités de l'ens des sommets de G

Prop: $\frac{1}{2} \text{diam}(G) \leq ecc(G) \leq \text{diam}(G)$

*preuve: * diam = + gde de toutes ecc en a ineq de ①. $ecc(v) \leq ecc(u)$
* $d(v, w) \leq d(v, u) + d(u, w) \leq 2ecc(u)$ $\forall v, w$ donc $\text{diam}(G) \leq 2ecc(u)$ d'où ineq ②*



II - Complexité distribuée:

II 1) Quelques mesures de complexité

• unité de temps?

modèles de cplx valables sous cd:

hypothèses: * les calculs sont instantanés

** pas d'erreur / panne / collision / désynchronisation*

* les communic (directes) sont unit^R (= l'unité de tps)

** id unique pour ent*

compte ps pq calc

Si $i = 0$ alors

Env k à 1 et à 2

Sinon

Rec k de 0

fin Si

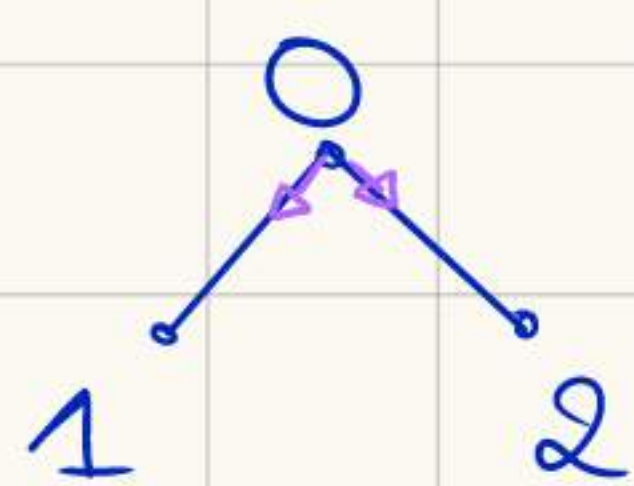
env à 1 et 2 = en m tps

Env k à 1

Env k à 2

Ex

$\text{msg} = 2$
 $\text{tps} = 1$



"échq"
de root
envoie +
récept°



cas général

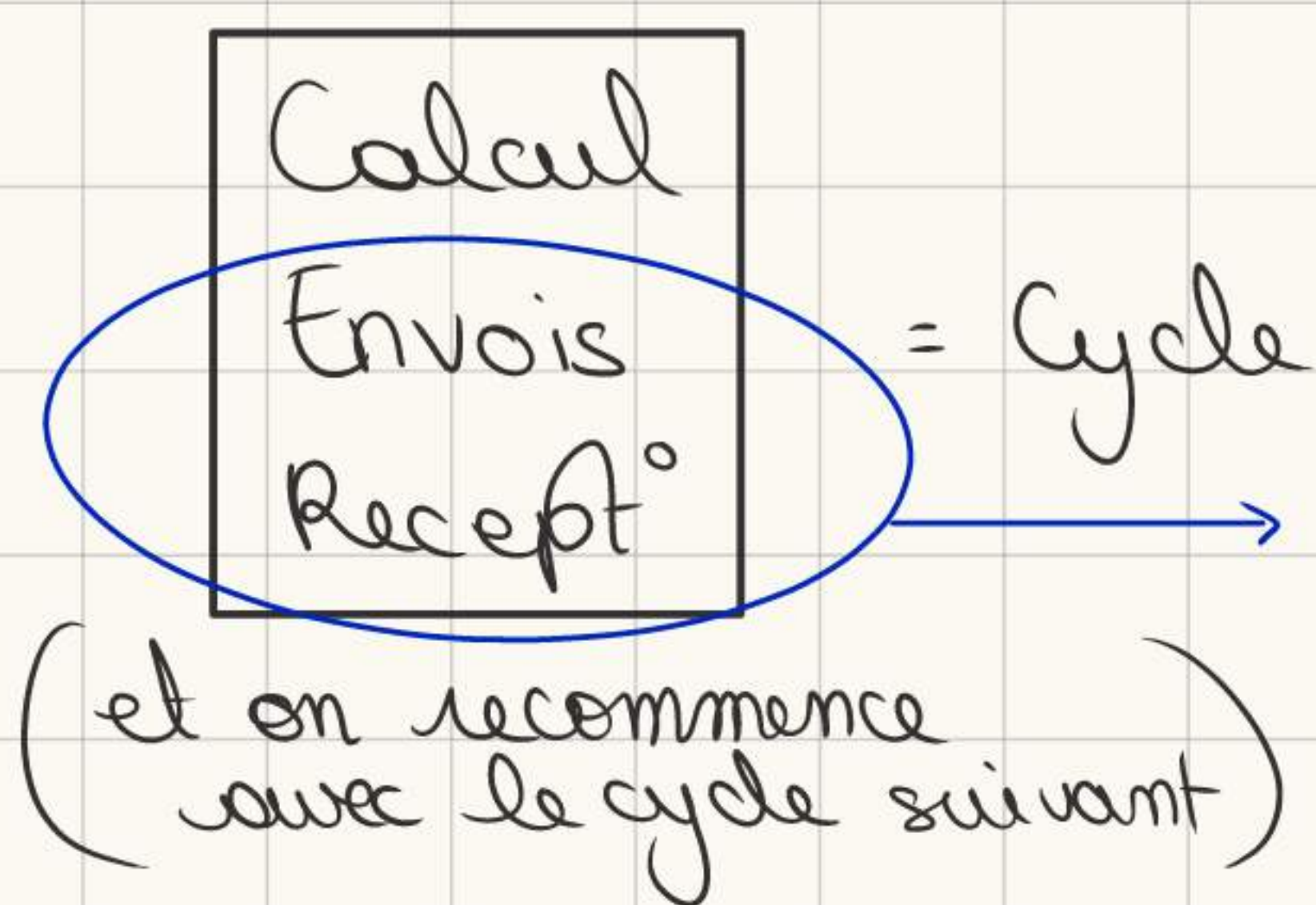
Def: La complexité en temps asynchrone d'un algo A distrib qui s'exé sur un syst dist de graphe G , notée $\text{TEMPS}(A, G)$ est le nb d'unités de tps qui s'écoule entre le déb et la fin de l'exé sous ces 2 hypothèses.

$\text{TEMP}(A, G) = \# \text{unités de tps needed à l'exé de } A \text{ sur } G \text{ ds le pire des cas, chq msg peut au pire 1 unité de tps pour traverser le lg d'une arête de } G$

Def: La complexité en messages d'un algo A distrib qui s'exé sur un syst dist de graphe G , notée $\text{MESSAGES}(A, G)$ est le nb de msg échangés lors de l'exécut de l'algo. *dans le pire des cas.*

Synchronisme

Le msg est découpé de la façon suivante :



$cplx^T$ synchrone on considère le tps de calcul !

l'algo progresse $\perp^t$ sur les $\neq$ entités de calculs sauf lors de l'attente d'un msg d'une autre entité
chq ent fait une suite de synchrones = cycles tq le cycle effectuée au m^{me} moment p^r tem

$\neq$ asynchrone

Def: la $cplx^T$ en tps synchrone d'un algo distribué A qui s'exécute sur un syst distrib de graphe G, noté $TEMPS(A, G)$, est le nb de cycles qui s'écoulent lors de l'exécution

needed à l'exé de A sur G de pire des cas

lim taille msg f^o nb d'ent

II 2 Modèles

- * Modèle local: pas de lim sur la taille des msg.
- * Modèle congest: la taille d'un msg ne pt pas dépasser $\log(n)$ en nb de bits ($O(\log(n))$) bits avec n = nb d'entités de calcul

* Modèle asynchrone

III - Problèmes distribués

Il s'agit de faire résoudre un pb (par ex calculer un $\oplus$ court chemin dans G) par les sommets du graphe!

ex de pb: * élect^o d'une ent qui aura rôle partic de calc
* diff
* autre courant

see Dijkstra

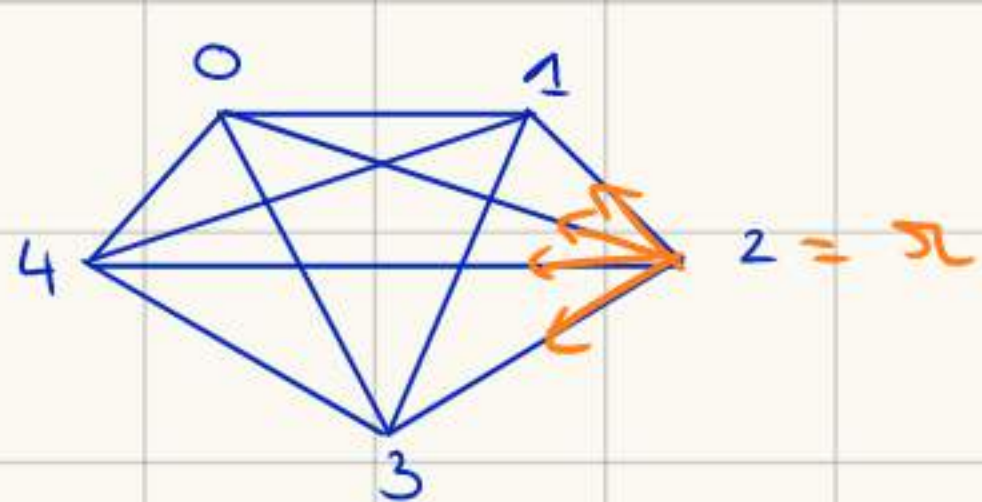
III 1 Premier problème diffusion

- * Une entité de calcul r (racine) possède une donnée (par ex un entier k) et doit la transmettre à toutes les autres entités de calcul.

dist ou indist

Ex Si le graphe G est complet :

Si $i == r$ alors



Envoyer k à toutes les autres entités de calcul

Sinon

Recevoir k de x

Fin si

$$\text{TEMPS}(A, K_n) = 1$$

graphe complet à n sommet

A alg de diffus° un G de n sommets, on a:

$$* \text{MESSAGE}(A, G) \geq n-1$$

$$* \text{TEMPS}(A, G) \geq \text{exc}(x)$$

preuve: * transmiss° = msg à 1 ent et y a $n-1$ ent à informé x
 H° direct als $n-1$ transmiss° de msg
 sinon @ $n-1$ donc hier au @ $n-1$
 * soit f ent à dist $\text{exc}_G(x)$
 et transmiss° d'1 ent à vous = dist f
 de need au @ 1 tps de $\text{exc}_G(x)$
 p° msg soit transmis de x à f

Que se passe-t-il si G n'est pas complet?

couvrant

Premier algo $\text{CAST}_T(x)$

graphe $\Rightarrow$ arbre couvrant $\Rightarrow$ ts les ent à m dist sont m hauteur

hauteur de l'arbre $T = \text{exc}_G(x)$

$$2^{H(k)}$$

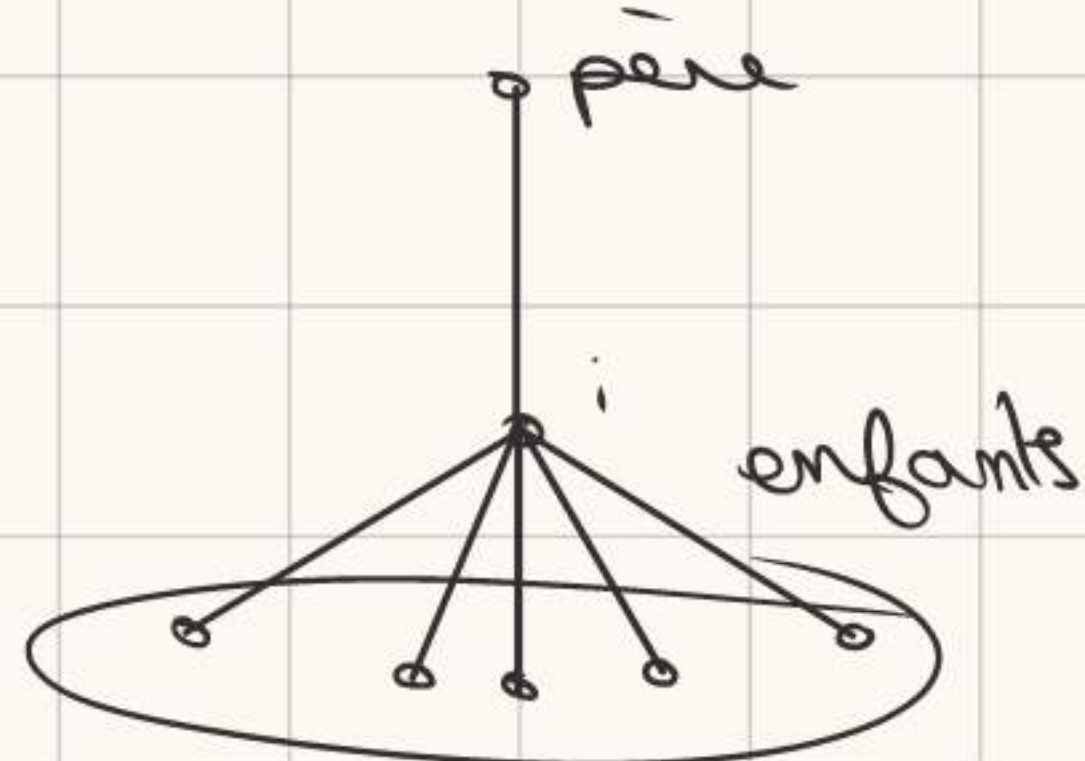
si chq ent coco père et liste fils
 als $\text{CAST} \Rightarrow$ rés. pb diffus° en $n-1$ msg

pas cycle, connexe, touche ts sommets

Ici T est un arbre couvrant de G de racine x

$\hookrightarrow$ l'entité i dispose de:

* une variable père qui vaut l'identifiant du père de i dans T



* un tableau enfants qui contient les identifiants des enfants de i dans T .

$\text{CAST}_T(x)$:

Si $i == x$ alors

Envoyer k à tous les enfants (de x)

Sinon

Recevoir k de père

Envoyer k à tous les enfants (de i)

Fin si

astg: easy to calc cplx° tps

Prop: arbre T a n sommets admet $n-1$ arêtes

preuve: induct°

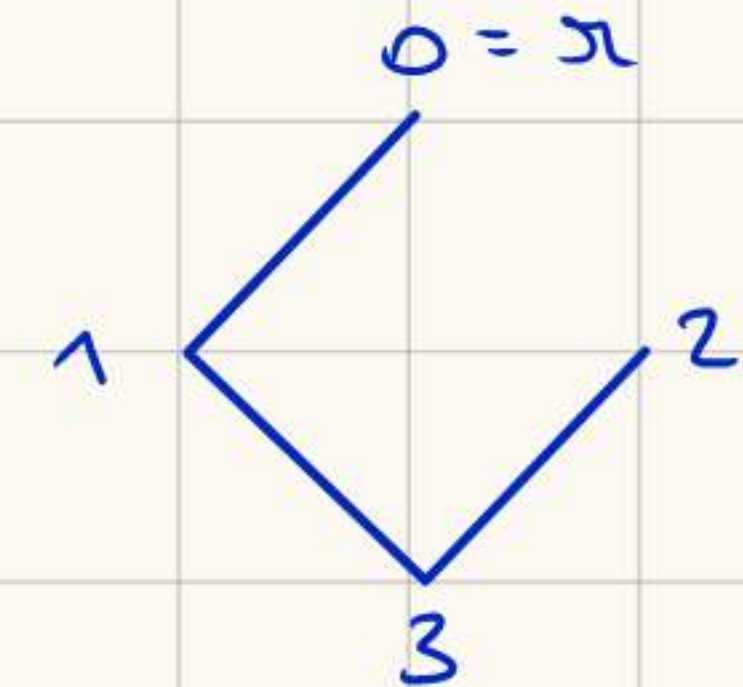
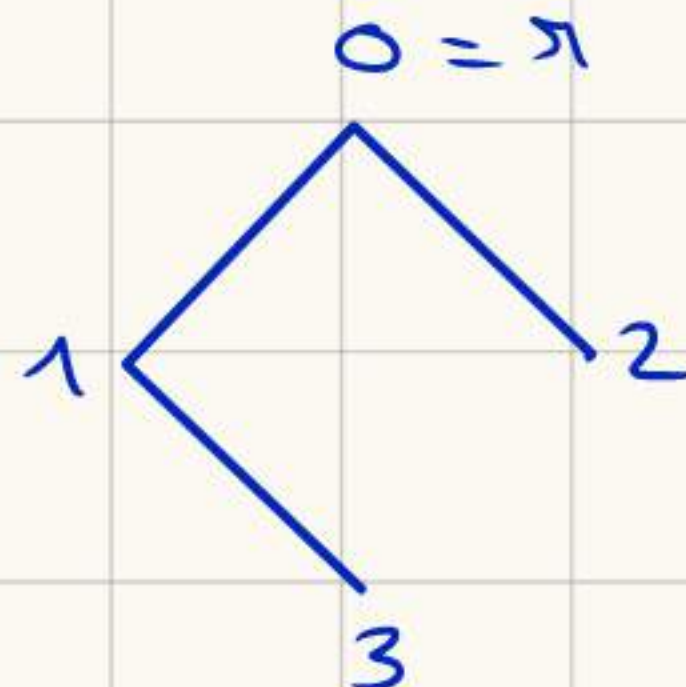
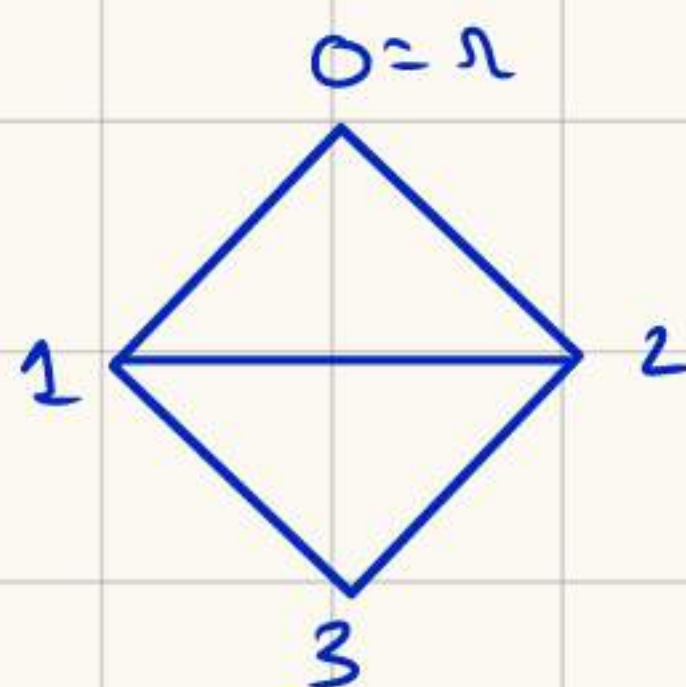
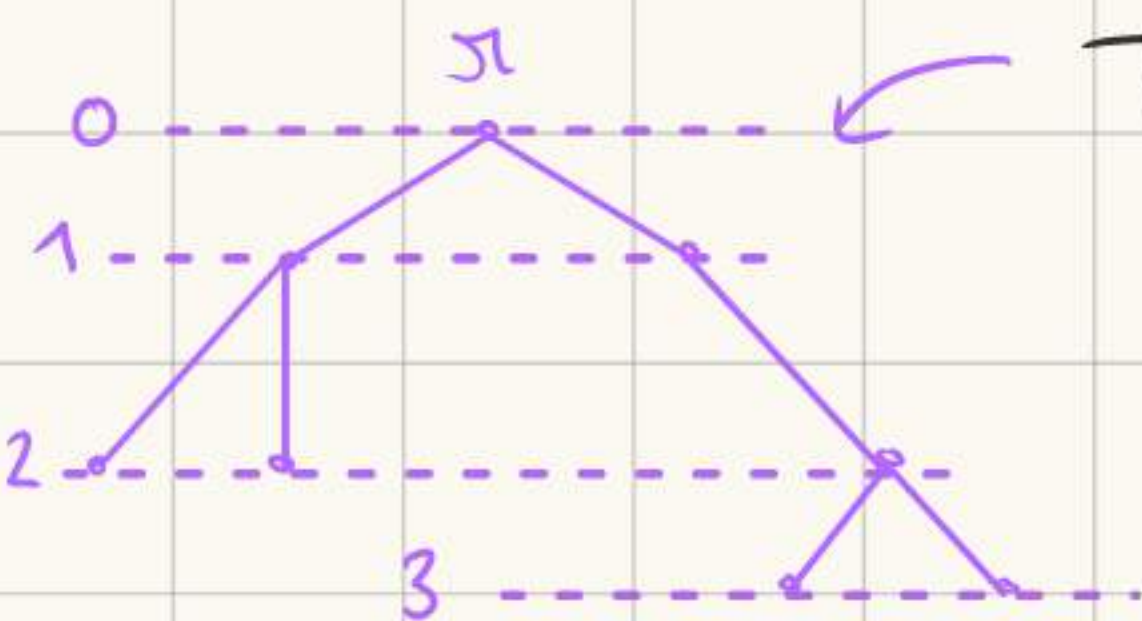
* $n=1$, 1 sommet $\Rightarrow$ 0 arête
 et $n-1 = 1-1 = 0$ c'qfd
 * $n>1$, # arêtes d'arbre à $n-1$ sommets = $n-2$

Pb: il ft d'abord calculer un arbre couvrant T .

Prop:

$$\text{MESSAGES}(\text{CAST}_T(x), G) = n-1 \quad (\text{pq } T \text{ a } n-1 \text{ arêtes})$$

$$\text{TEMPS}(\text{CAST}_T(x), G) = \text{hauteur de } T$$



$$\text{hauteur}(T_1) = 2$$

$$\text{hauteur}(T_2) = 3$$

si pas d'arbre couvrant $\Rightarrow$ technq inondat° = FLOOD(x)

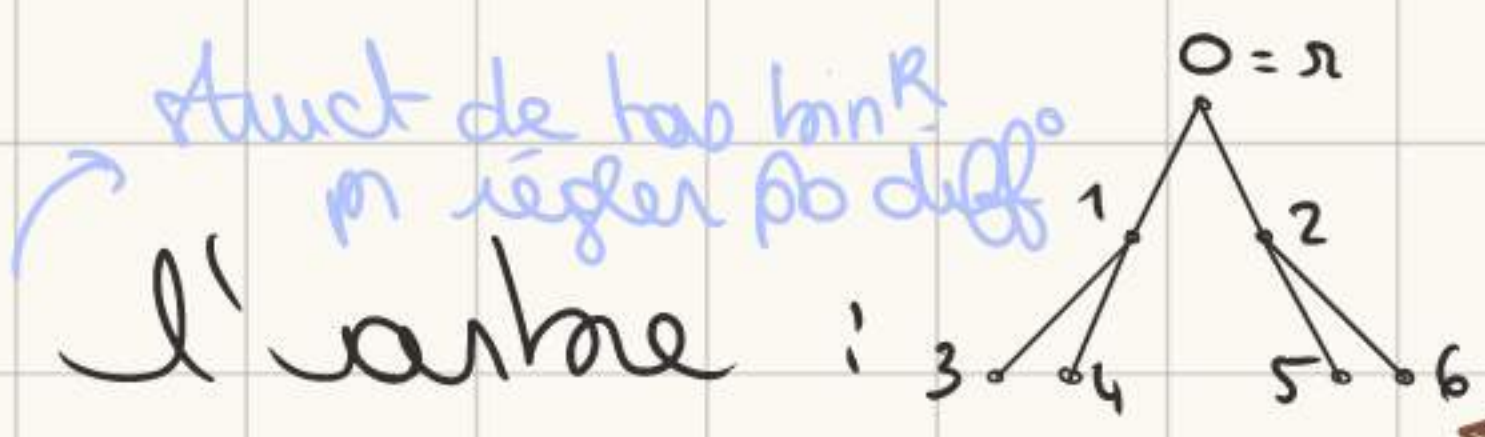
Cas particulier:

Si $\hookrightarrow$ bas binaire de graphe complet

* G est complet

* les sommets de G sont numérotés de 0 à $n-1$.

* $n = 0$

On pt voir l'arbre :  structure de bas binaire

les enfants de i sont : $\begin{cases} 2i+1 \\ 2i+2 \end{cases}$

le père de i (>0) est $\lfloor (i-1)/2 \rfloor$
 on cela on ?

Exo : adapter le pseudo-code de $CAST_T(0)$ à cet arbre

$CAST_T(0)$:

Si $i == 0$ alors

Envoyer k à 1 et 2

Sinon

Tant que $i < n$

Si $i \% 2 = 1$

Recevoir k de père

Envoyer k à $i+1$ et $i+2$

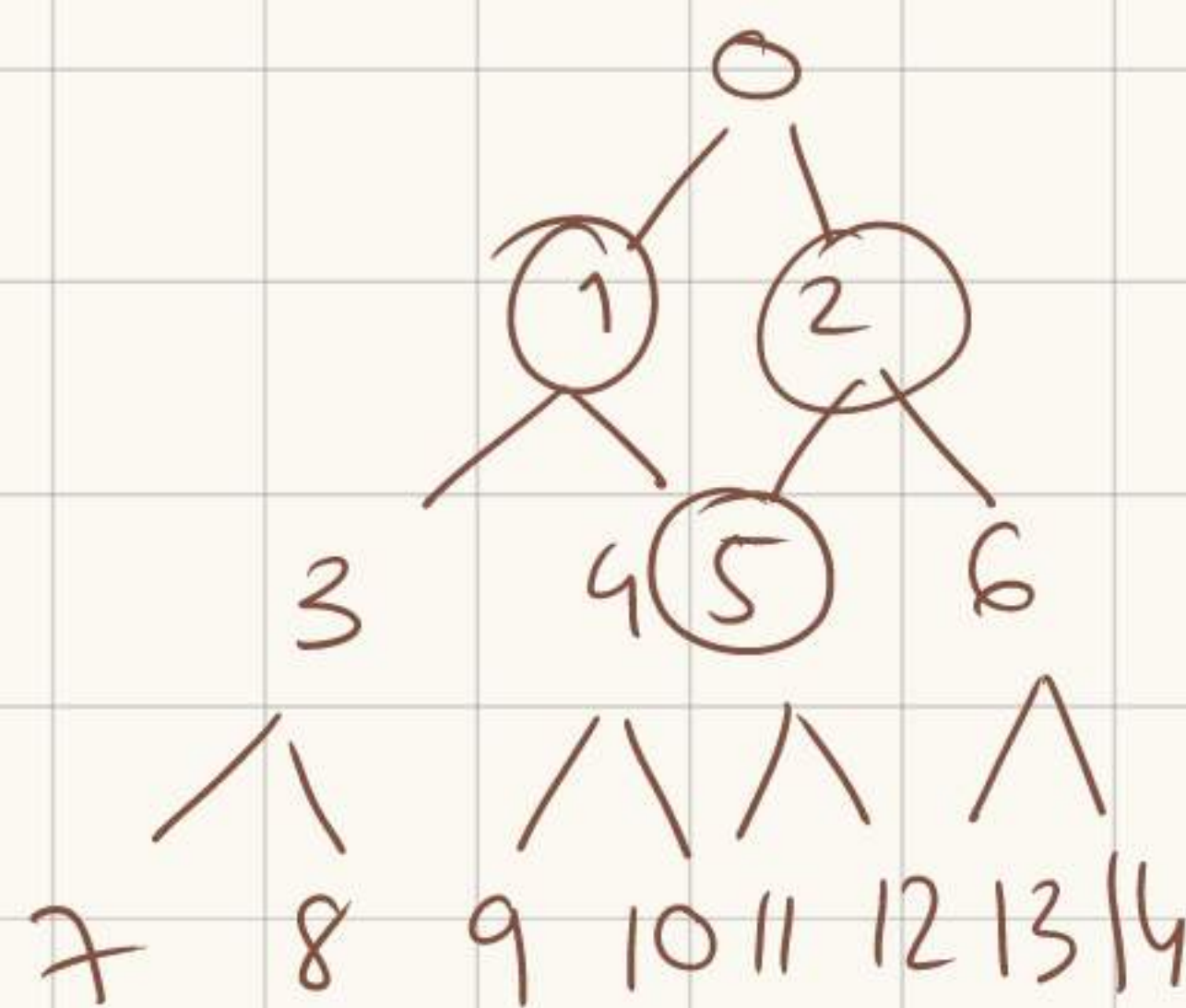
Si $i \% 2 = 0$

Recevoir k de père

Envoyer k à $i+3$ et $i+4$

fin tant que

fin si



Correction:

Si $i \neq 0$ alors

Rec k de $\lfloor (i-1)/2 \rfloor$

fin si

Si $2i+1 < n$

Envoyer k à $2i+1$

fin si

Si $2i+2 < n$

Envoyer k à $2i+2$

fin si

hauteur de l'arbre:
 $\log_2(n)$

par si on a arbre binaire parfait, y a

hauteur

var = exponent $\Rightarrow$ expo
 ici $\Rightarrow \log$

$\Leftrightarrow$ hauteur = $\log_2(\# \text{ nœuds})$

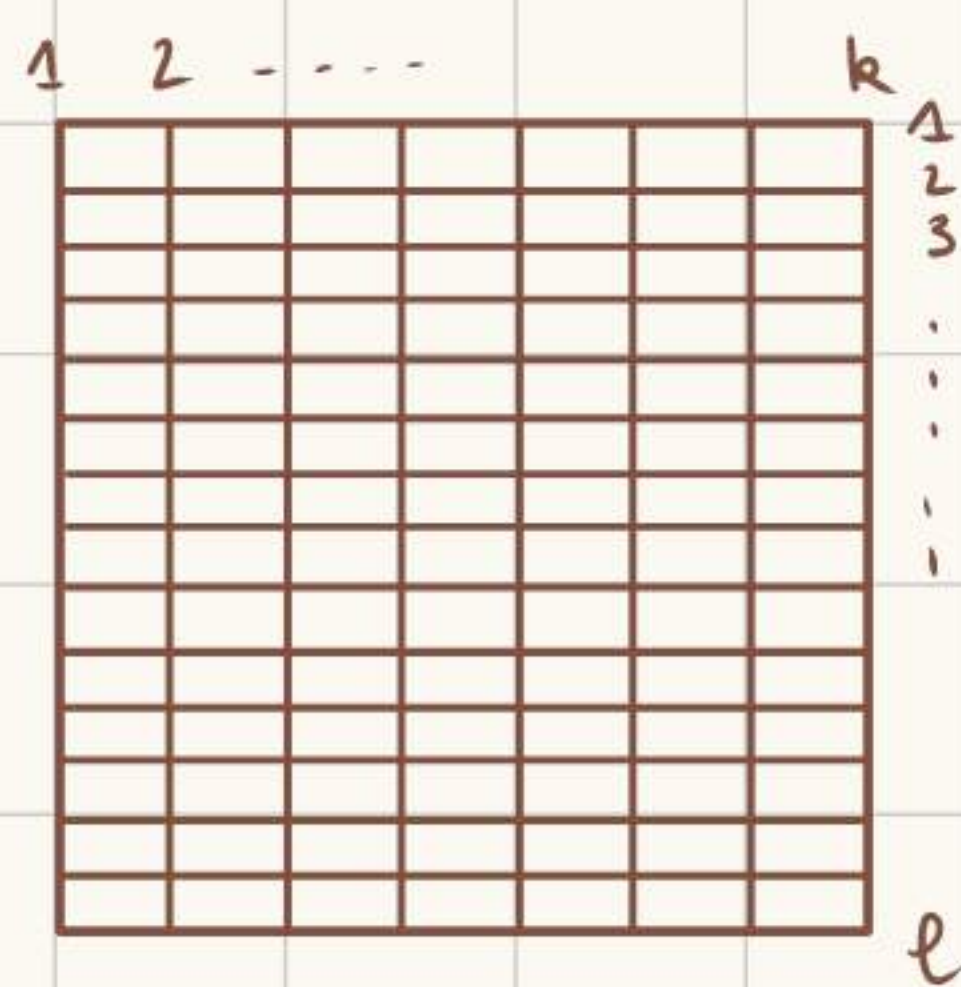
Exo

cycle $\rightarrow$ ds ces cas
 graphe complet $\rightarrow$ tous les pts = centres
 chemin pair $\rightarrow$ $\approx$ symétrie

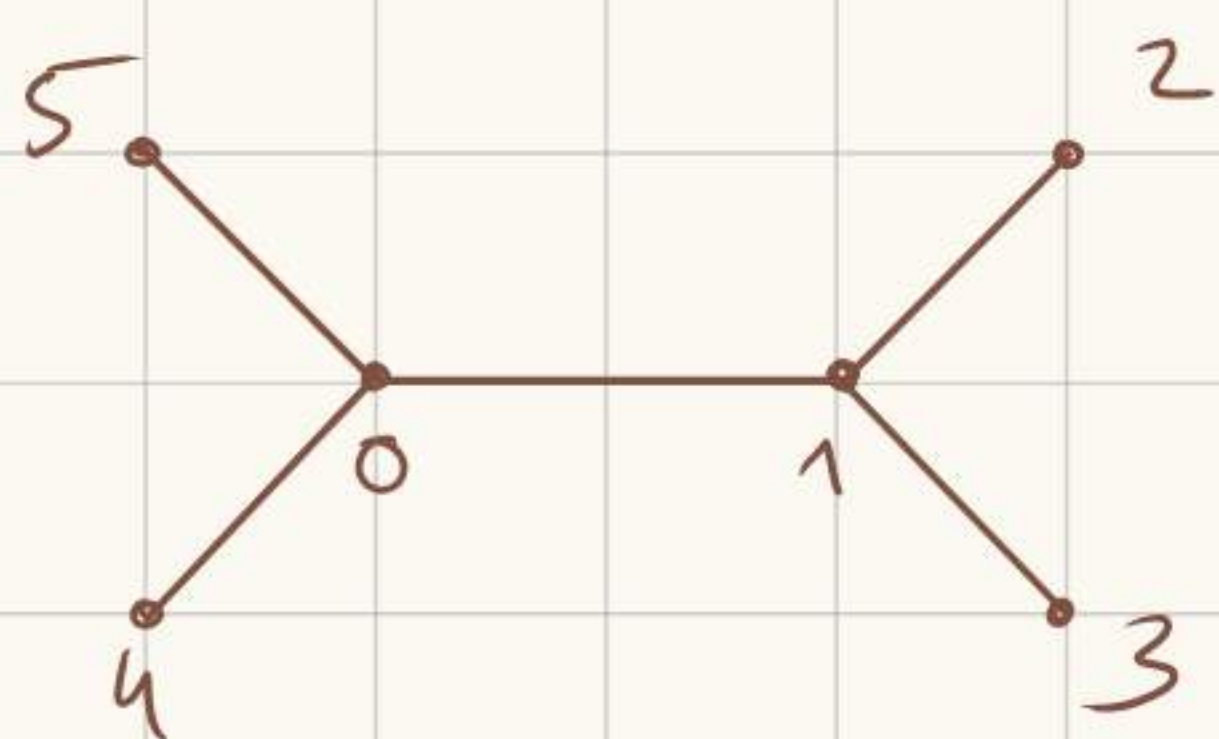
(a) Trouver un graphe G avec plus centres

(b) Calculer $\text{diam}(G)$ et $\text{ecc}(G)$ lorsque G est :

- 1) un chemin
- 2) un cycle
- 3) un graphe complet
- 4) une grille $k \times l$:

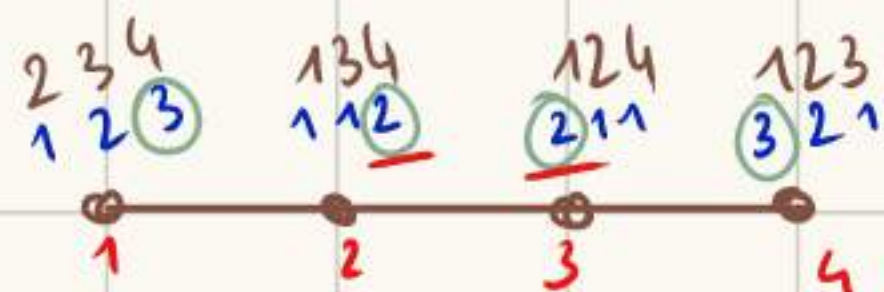


(a)



(b)

(1)

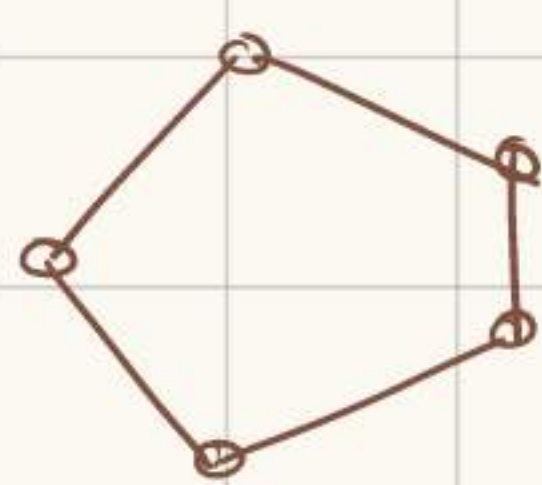


$$\text{diam}(G) = n - 1$$

$$\text{ecc}(G) = 2 \text{ et centre } G = \{2, 3\}$$

$$\hat{m} \text{ ecc pts} \Rightarrow \text{ecc } G = \text{diam } G$$

(2)



$$\text{diam}(G) = \parallel$$

$$\text{ecc}(G) = \lfloor n/2 \rfloor$$

(a) Dans le graphe complet sur n sommets, (avec $n \geq 2$ sinon y a 1 seul pt = 1 seul centre), tous les sommets ont une excentricité de 1 et donc le centre du graphe complet est en fait (tous) ses n sommets.

(b)

(1)

cas chemin impair : $\rightarrow \frac{n-1}{2} = \lfloor \frac{n}{2} \rfloor$ car n est impair

cas chemin pair : $\rightarrow \frac{n}{2}$

Appelons n le nb de sommets d'un chemin simple P et raisonnons en f° de la parité de n :

* n est impair :

The diagram shows a horizontal line with points marked by dots. Below the line, the points are labeled with numbers. The sequence of labels is: 1, 2, 3, followed by an ellipsis, then $\frac{n-1}{2}$, $\frac{n+1}{2}$, $\frac{n+3}{2}$, followed by another ellipsis, then $n-2$, $n-1$, and n . The point labeled $\frac{n+1}{2}$ is marked with a red dot, while all other points are marked with green dots.

$$\text{ecc}(G) = \text{ecc}_G\left(\frac{n+1}{2}\right) = \frac{n-1}{2} = \left\lfloor \frac{n}{2} \right\rfloor$$

* n est pair :

$$\text{ecc}(G) = \text{ecc}_G\left(\frac{n}{2}\right) = \text{ecc}_G\left(\frac{n}{2} + 1\right) = \frac{n}{2} \left(= \left\lfloor \frac{n}{2} \right\rfloor \right)$$

Par crot : $\text{ecc}(G) = \left\lfloor \frac{n}{2} \right\rfloor$

Et le diam dans les deux cas est $\text{ed}(1, n) = n - 1$.

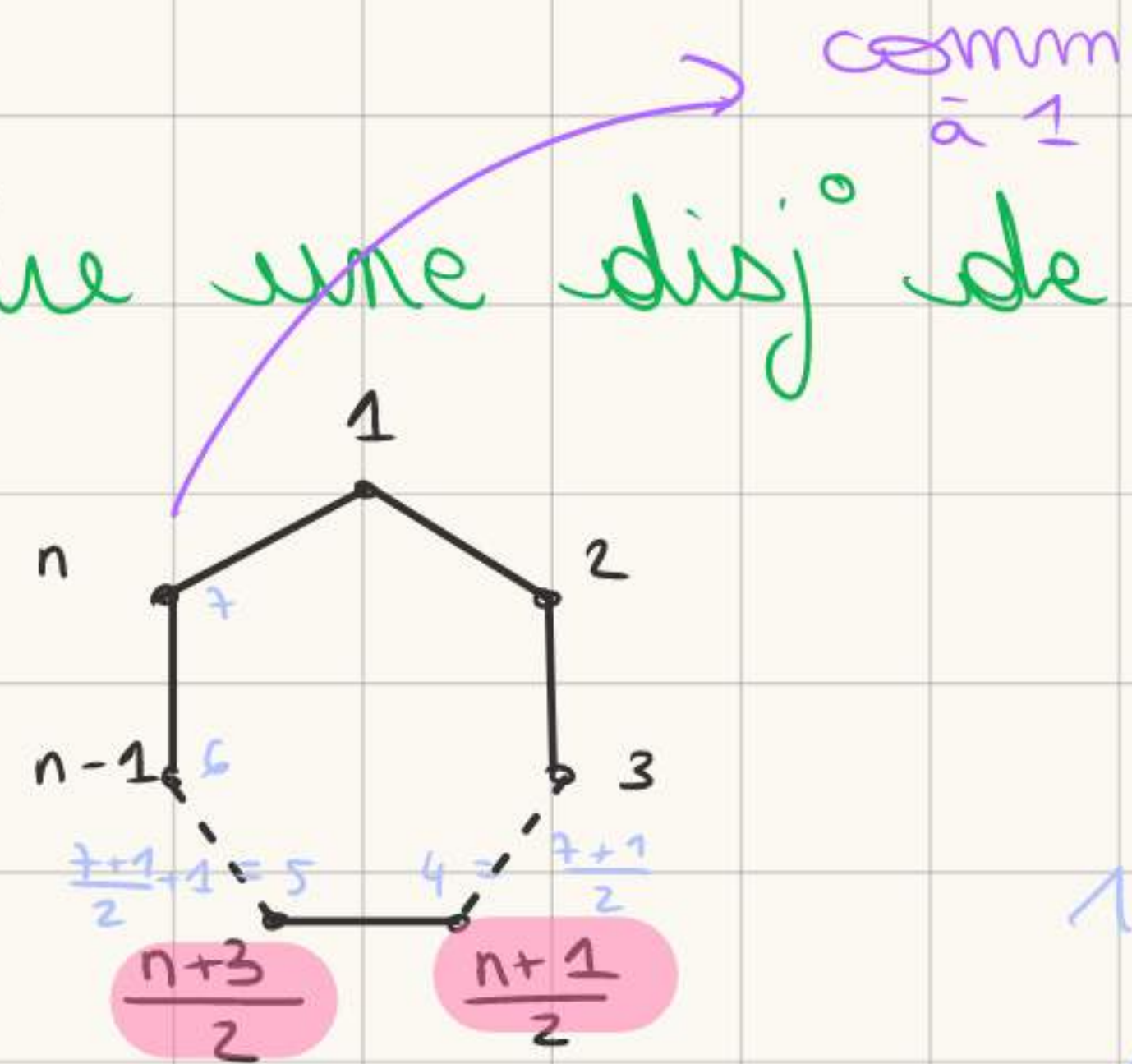
② Là aussi, nous allons faire une disj^o de cas en f^o du nb n de sommets:

- * si n est impair:



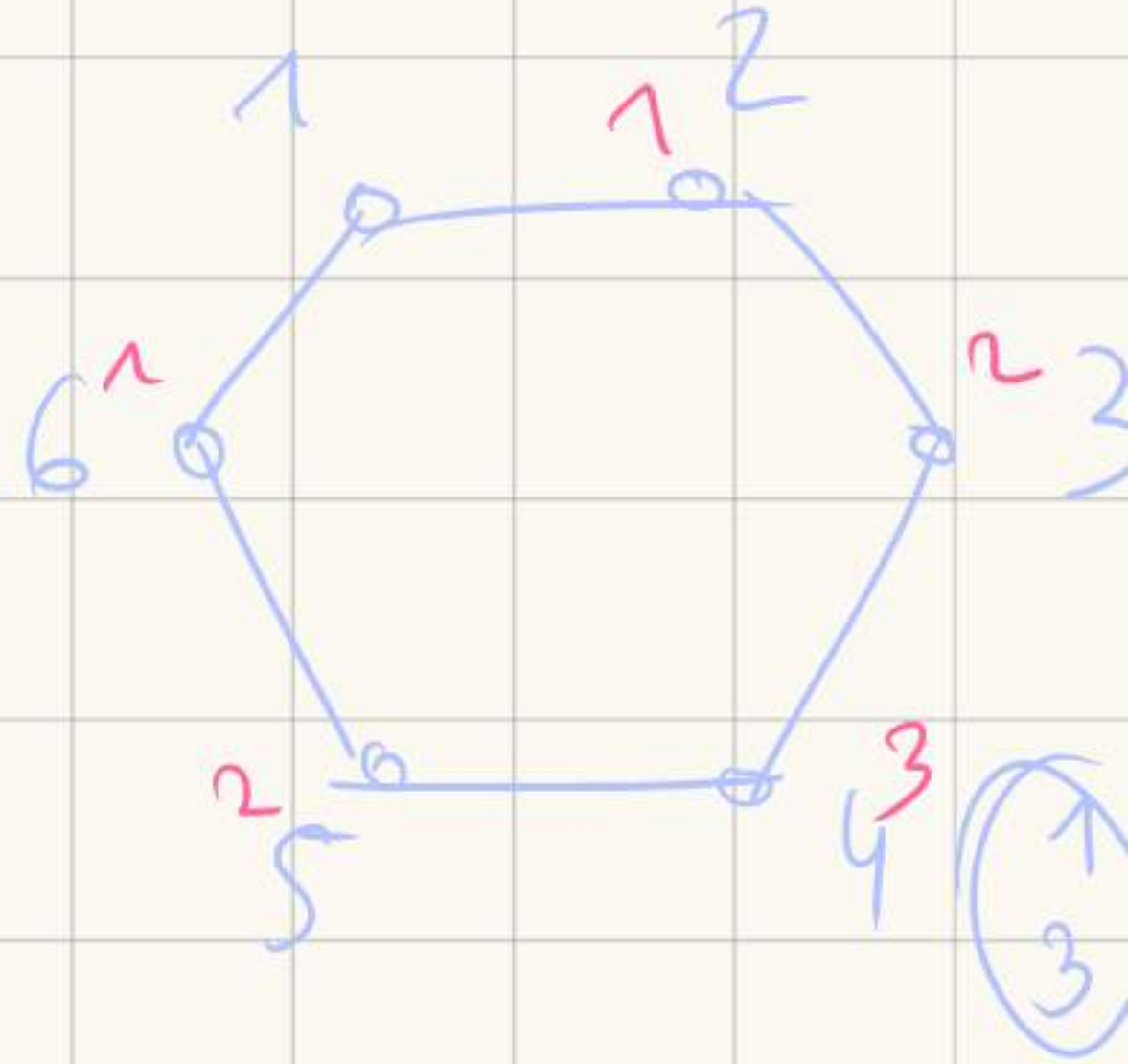
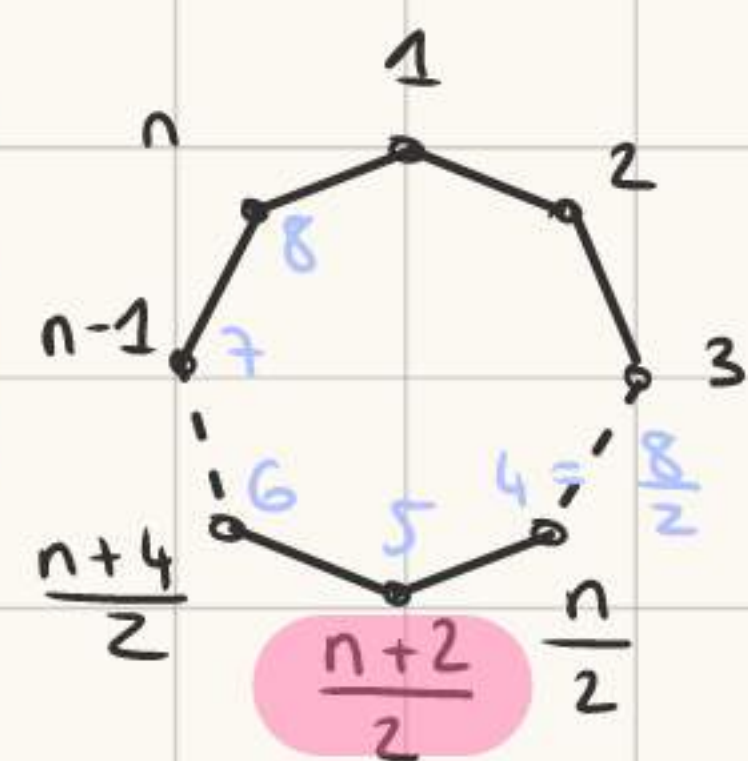
* si n est impair:

$$\text{vec}_G(1) = \frac{n-1}{2} = \lfloor n/2 \rfloor$$



* si n est pair :

$$\text{sec}_G(1) = \frac{n}{2} = \left\lfloor \frac{n}{2} \right\rfloor$$



Puisq les des sommets ont la m même excentricité, on obtient $\text{ecc}(G) = \lfloor \frac{n}{2} \rfloor$ et $\text{diam}(G) = \text{ecc}(G) = \lfloor \frac{n}{2} \rfloor$

en effet, $\text{diam}(G) = \max_{u,v \in V} \{d(u,v)\} = \max_{u \in V} \underbrace{\left\{ \max_{v \in V} \{d(u,v)\} \right\}}_{\text{ecc}}$

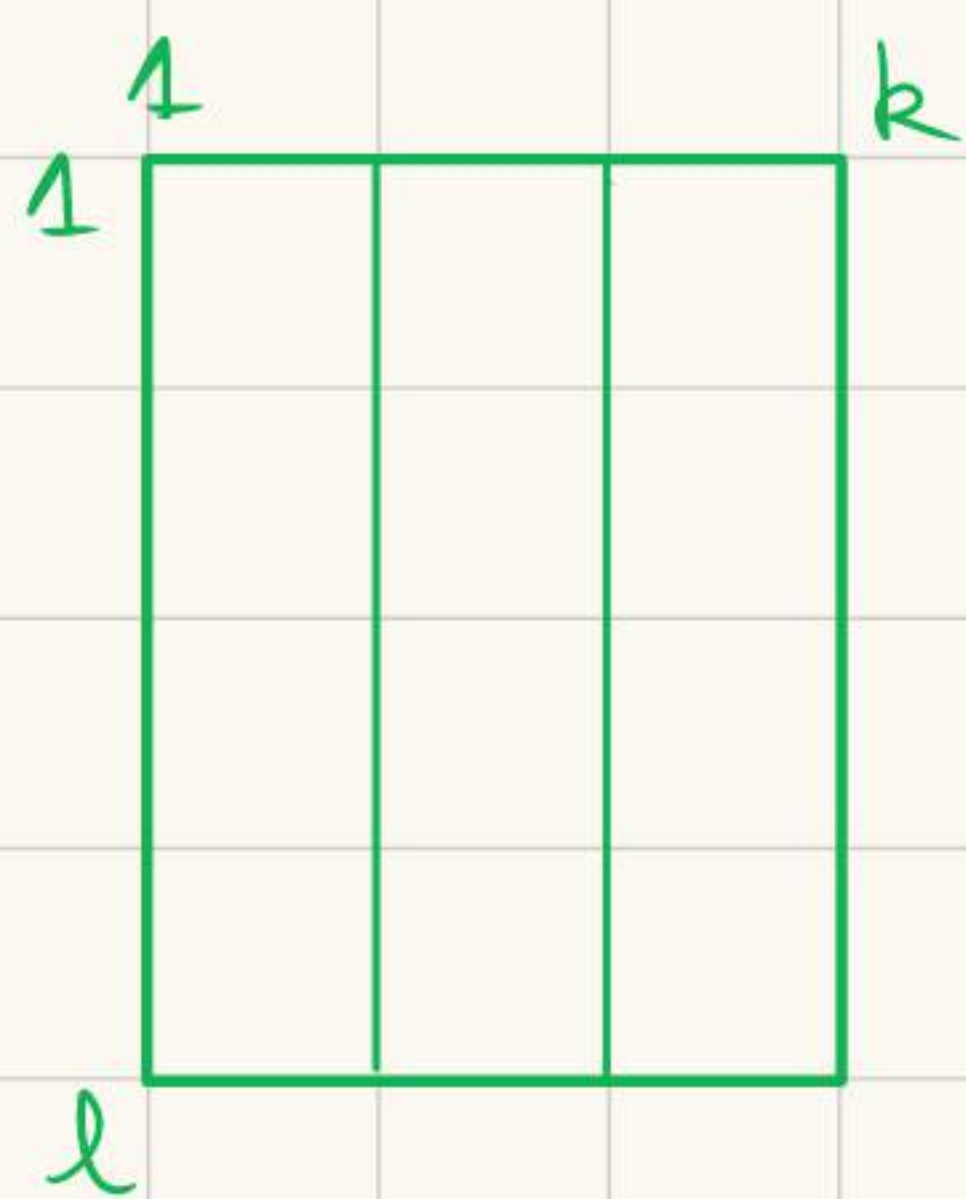
graphe complet
 $\Rightarrow \text{ecc}(G) = \text{diam}(G) = 1$

$$= \max_{u \in V} \{ \text{ecc}_G(u) \} = \min_{u \in V} \{ \text{ecc}_G(u) \} = \text{ecc}(G)$$

pcq tous les sommets
ont la m[^]e excentricité

gille : puisq cette gille est le prod cart d'un chem¹
simple avec k sommets et d'un autre avec l

sommets :



$$\text{ecc}(G) = \left\lfloor \frac{k}{2} \right\rfloor + \left\lfloor \frac{l}{2} \right\rfloor \quad (\text{d'après b1})$$

$$\text{diam}(G) = k-1 + l-1 \quad (\text{d'après b1})$$

19/02

Diffusion

- * Une entité de calcul α (= racine) dispose d'une info k
- * Elle doit envoyer k à toutes les autres entités

→ 1^{er} algo : $\text{CAST}_T(\alpha)$ → ici T est un EP recourant du graphe du syst dist.

En pratique : chq entité i a :

- * une variable père = identifiant de père i dans T
- * un tableau enfants qui cont les identifiants de tous les enfants de i ds T .
- * une variable nb-enfants qui indique le nb d'enfants de i dans T

q° : $\hat{C}^t$ on construit un EP recourant?

→ réponse : 2^e algo : Flood (α)

Si $i = \alpha$ alors

Env k à tous les voisins (de α) ds G

Si non

→ on sait que c'est k car k est le seul msg envoyé de H le code
→ ça va arriver car G connexe ($\hat{C}^t$ jrs)

Rec k d'une entité de calcul (un voisin), n'imp pas

Poser père ← identifiant de l'envoyeur du msg reçu.

Env k à tous les voisins (de i) ds G (sauf père)

Fin si

Proposé d'algo :

① Env "tu es mon père" à père

~~1~~ ≠ optR

Pour j allant de 1 à nb-enfants faire
Rec "tu es mon père" d'une entité de calcul
Mettre l'identifiant de l'envoyeur (n'importe lequel) dans le tableau enfants.

?

Thm: On ne pt pas coco notre nb d'enfants avant de
f^R ces recept°.

Stratégie: lorsqu'on a besoin d'info prov^t de nos voisins
mais qu'on ne coco pas le nb de voisins qui doivent
nous l'envoyer, on reçoit un msg de tous les voisins
et ce msg contiendra 1 ou 0 selon que ce voisin devrait
nous envoyer (ou pas) l'info voulue.

pas
balise
vmt cont
du msg

Flood & Echo(π) → construit le père et le tab enfants

Si $i = \pi$ alors

Env ($k, 0$) à tous les voisins (de π)

Poser $c \leftarrow (\# \text{ voisins})$ // c sera le # de recept° à f^R

Sinon

Rec ($k, 0$) d'un voisin père (inconnu)

Env ($k, 0$) à tous les voisins (de i) sauf père

Poser $c \leftarrow (\# \text{ voisins} - 1)$

Fin Si

Tant que $c > 0$ faire

Rec ($m, 1$) d'un voisin j (inconnu)

Si $m = 1$ alors

Ajouter j au tableau enfants

Poser nb-enfants $\leftarrow$ nb-enfants + 1

Fin Si

Poser $c \leftarrow c - 1$

Fin tant que

As: pt rec encore

de balise 0

et ft send balise 1.

Stratégie: ↴

Si $i == r$ alors

Env $(k, 0)$ à tous les voisins (de r)

Poser $c \leftarrow (\# \text{ voisins})$ // c sera le # de recept° à f^R

Sinon

Rec $(k, 0)$ d'un voisin père (inconnu)

Env $(k, 0)$ à tous les voisins (de i) sauf père

Poser $c \leftarrow (\# \text{ voisins} - 1)$

fin si

Tant que $c > 0$ faire

Rec (m, b) d'un voisin j (inconnu)

Si $b == 1$ alors

Si $m == 1$ alors

Ajouter j au tableau - enfants

Poser nb-enfants $\leftarrow$ nb-enfants + 1

fin si

Poser $c \leftarrow c - 1$

fin si

Si $b == 0$

Env $(0, 1)$ à j :

fin si

fin Tant que

Si $i \neq r$ alors:

Env $(1, 1)$ à père

fin si

voir algo
Bellman Ford

pb: compteur

Stratégie: ↴

Si $i == r$ alors

donnée à diffuser

Env $(k, 0)$ à tous les voisins (de r)

Poser $c \leftarrow 2(\# \text{ voisins})$ // c sera le # de recept° à f^R !

Sinon

Rec $(k, 0)$ d'un voisin père (inconnu)

Env $(k, 0)$ à tous les voisins (de i) sauf père

si racine,
+ env donnée à
+ ses voisins
+ att un tableau
de chacun
* NPI
⇒ 2 retours par lien
⇒ $c_{tot} \times 2$

rec donnée de père
et l'env à ts ses
autres vois
⇒ $c_{tot} = 2(\deg - 1)$

$\# \text{vois} - 2$
 Poser $c \leftarrow 2(\# \text{voisins} - 1) = c \leftarrow 2(\# \text{vois}) - 2 = 2 \deg(u) - 2$
 fin si
 Tant que $c > 0$ faire
 Rec (m, b) d'un voisin y (inconnu)
 si $b == 0 \rightarrow (m, 0) = \text{donnée à diffuser}$
 Env $(0, 1)$ à y
 Poser $c \leftarrow c - 1$ on $\downarrow$ notre propre cpt
 fin si
 si $b == 1$ alors $(m, 1)$
 si $m == 1$ alors $(1, 1)$ on rec "acc recept $\oplus$ je vx bien que tu sois mon père"
 Ajouter y au tableau - enfants
 Poser nb-enfants $\leftarrow$ nb-enfants + 1
 Poser $c \leftarrow c - 2$ permet la dtd d'ent si on dev père
 fin si
 si $m == 0$
 Poser $c \leftarrow c - 1$
 fin si
 fin Tant que
 Si $i \neq r$ alors:
 Env $(1, 1)$ à père
 Rec "fin" de père
 fin si
 ent a fini $\leftarrow$ Poser fin(i) $\leftarrow 1$
 Pour chq j dans tab-enfants faire:
 Env "fin" à j
 fin Pour

boucle d'att: nœud bloque ds boucle tant que pas reçu ts ses 2 msg de ses enf
 rec d'une donnée als que ya déjà 1 père donc env acc-rec (0,1)
 (k,0), (m,0) = donnée
 (0,1), (1,1) = acc-rec
 père
 père
 recept° d'un acc recept père = j veut de nous c père donc add j à notre liste enf
 on décr juste $\leftarrow$ en rec: acc rec ms j a déjà un père
 terminaison $\leftarrow$ on y entre qd $c=0 \Rightarrow$ nœud sait que ts ses voisins ont fini
 on att autoris du père pr finir
 ent a fini $\leftarrow$ Propag° msg de fin vers le bas de l'ar

Problème de la concentration

- * toutes les ent de calculs ont une donnée k .
- * l'ent r doit récupérer les val de k de toutes les autres (ds un tableau)

Hypothèse : on a déjà construit (par ex avec Flood & Echo)
 un arbre recourant T (ou m même la var père)

Si $i == r$ alors
 Rec k de tous les enfants
 Sinon
 Rec k de tous les enfants
 Env le tableau obtenu à père
 Fin Si

Les envoient un tab.
 pas bloq. pr feuille car feuille n'a pas d'enf. donc cette ligne skipped pr feuille

Problème de la réduction

Si $i == r$
 Pour $j = 1$ à nb-enfants
 Rec k' d'un enfant
 $k \leftarrow op(k, k')$ —————> où op est une opérat^o associative
 Fin Pour
 Sinon
 Pour $j = 1$ à nb-enfants
 Rec k' d'un enfant
 $k \leftarrow op(k, k')$
 Fin Pour
 Env k à père
 Fin Si

Gauss Jordan

$$Ax = b$$

$$x_1 + x_2 = 1$$

$$x_1 - x_2 = 3$$

$$A = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \quad b = \begin{bmatrix} 1 \\ 3 \end{bmatrix} \quad x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

→ = l'ens des vect sol° qui vérifient

Gauss Jordan donne l'ens S des vecteurs x tq $Ax = b$

$$A = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \vdots & & \vdots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \quad b = \begin{bmatrix} b_1 \\ \vdots \\ b_m \end{bmatrix} \quad x = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

(n et m arbitraires)

$$S = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = v^0 + \lambda_1 v^1 + \dots + \lambda_k v^k : \lambda_1, \dots, \lambda_k \in \mathbb{R}$$

$$v^i = \begin{bmatrix} v_1^i \\ \vdots \\ v_n^i \end{bmatrix}$$

$$\begin{aligned} x_1 + x_2 + x_3 + x_4 + x_5 &= 3 \\ x_1 + 2x_2 - x_3 - x_5 &= 2 \\ 2x_1 - x_2 + x_3 - x_4 &= 4 \\ 3x_1 + x_2 + 2x_3 - x_4 - x_5 &= 6 \end{aligned}$$

on pt pas avoir mat identité pcq pas $\square$ ms on pt rés.

$$\begin{array}{ccccc|c} 1 & 1 & 1 & 1 & 1 & 3 \\ 1 & 2 & -1 & 0 & -1 & 2 \\ 2 & -1 & 1 & -1 & 0 & 4 \\ 3 & 1 & 2 & -1 & -1 & 6 \end{array}$$

$$l_2 \leftarrow l_2 - l_1$$

$$l_3 \leftarrow l_3 - 2l_1$$

$$l_4 \leftarrow l_4 - 3l_1$$

→ ça mat augmentée

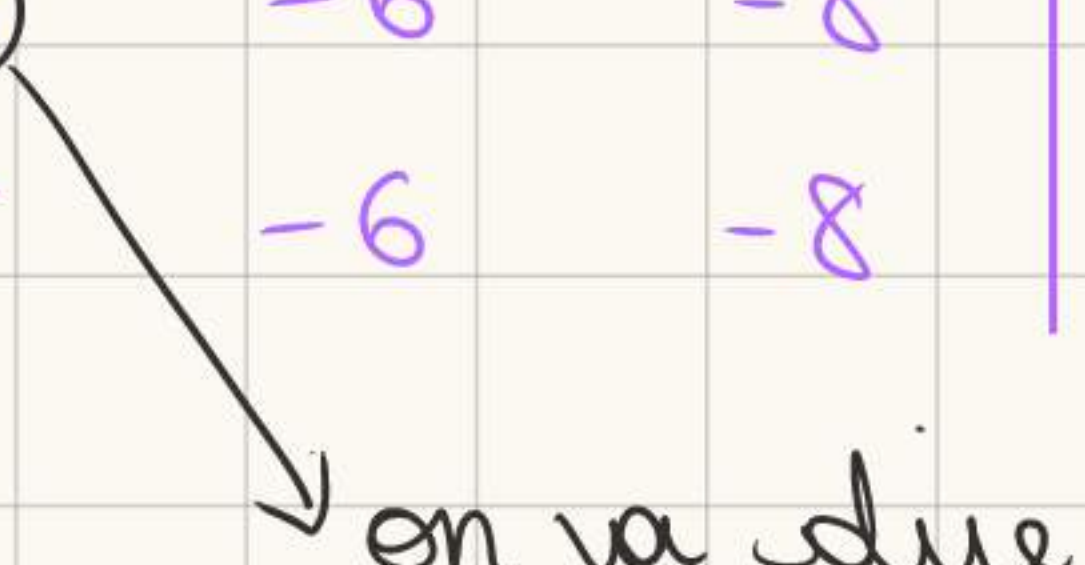
$$\begin{array}{ccccc|c}
 \overbrace{}^A & & \overbrace{}^b & & \\
 1 & 1 & 1 & 1 & 1 & 3 & l_1 \leftarrow l_1 - l_2 \\
 0 & 1 & -2 & -1 & -2 & -1 & \\
 0 & -3 & -1 & -3 & -2 & -2 & l_3 \leftarrow l_3 + 3l_2 \\
 0 & -2 & -1 & -4 & -4 & -3 & l_4 \leftarrow l_4 + 2l_2
 \end{array}$$

① Choix du pivot : parcourt 1^{er} élé de 1 col jusqu' $\neq 0$
 si tous $= 0$ on passe à col d'après
 puis on parcourt toute la col jusqu' $\neq 0$ etc.
 1^{er} $\neq 0$ on échg les lignes.

2 variables : ligne du pivot
 col (j) $\Rightarrow$ au déb 1,1.

② Après un pivot, on augmente l de 1
 (si pas de pivot, toute la col = 0, on laisse l tranquille)
 entouré = pivot

$$\begin{array}{ccccc|c}
 1 & 0 & 3 & 2 & 3 & 4 \\
 0 & 1 & -2 & -1 & -2 & -1 \\
 0 & 0 & \textcircled{-5} & -6 & -8 & -5 & l_3 \leftarrow \frac{l_3}{-5} \\
 0 & 0 & -5 & -6 & -8 & -5
 \end{array}$$


 en va due



③ On ne pt pivoter que sur un $\textcircled{1}$ d'où

$$\begin{array}{ccccc|c}
 1 & 0 & 3 & 2 & 3 & 4 \\
 0 & 1 & -2 & -1 & -2 & -1 \\
 0 & 0 & 1 & 6/5 & 8/5 & 1 \\
 0 & 0 & 0 & 0 & 0 & -5
 \end{array}$$

autre ex:

$$\begin{array}{ccccc|c} \textcircled{1} & -1 & 4 & 1 & 2 & -5 \\ 0 & 1 & -3 & 1 & 2 & 3 \\ 2 & 1 & -1 & 2 & 7 & -1 \\ -1 & -1 & 2 & 0 & -4 & 0 \end{array} \quad l_3 \leftarrow l_3 - 2l_1$$

$$\begin{array}{ccccc|c} 1 & -1 & 4 & 1 & 2 & -5 \\ 0 & \textcircled{1} & -3 & 1 & 2 & 3 \\ 0 & 3 & -9 & 0 & 3 & 9 \\ 0 & -2 & 6 & 1 & -2 & -5 \end{array} \quad \begin{array}{l} l_1 \leftarrow l_1 + l_2 \\ l_3 \leftarrow l_3 - 3l_2 \\ l_4 \leftarrow l_4 + 2l_2 \end{array}$$

ici on a les 1 du (sur le 1 en a pu)
on a des 0 donc décale à droite
sans changer ligne (2) de
les 2 sont pas forcés sur d'ég
mo autres

$$\begin{array}{ccccc|c} 1 & 0 & 1 & 2 & 4 & -2 \\ 0 & 1 & -3 & 1 & 2 & 3 \\ 0 & 0 & 0 & \textcircled{-3} & -3 & 0 \\ 0 & 0 & 0 & 3 & 2 & 1 \end{array} \quad l_3 \leftarrow l_3 / (-3)$$

$$\begin{array}{ccccc|c} 1 & 0 & 1 & 2 & 4 & -2 \\ 0 & 1 & -3 & 1 & 2 & 3 \\ 0 & 0 & 0 & \textcircled{1} & 1 & 0 \\ 0 & 0 & 0 & 3 & 2 & 1 \end{array} \quad \begin{array}{l} l_1 \leftarrow l_1 - 2l_3 \\ l_2 \leftarrow l_2 - l_3 \\ l_4 \leftarrow l_4 - 3l_3 \end{array}$$

$$\begin{array}{ccccc|c} 1 & 0 & 1 & 0 & 2 & -2 \\ 0 & 1 & -3 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & \textcircled{-1} & 1 \end{array} \quad l_4 \leftarrow l_4 / (-1)$$

$$\begin{array}{ccccc|c} 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & -3 & 0 & 0 & 4 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & \textcircled{1} & -1 \end{array} \quad l_3 \leftarrow l_3 - l_4$$

1	0	1	0	0	0
0	1	-3	0	0	4
0	0	0	1	0	2
0	0	0	0	1	-1

= mat augmentée échelonnée réduite

Poser $l \leftarrow 1$ → est

Pour j allant de 1 à n faire:

Poser $i \leftarrow l$

Tant que $i \leq m$ et $a_{ij} = 0$ faire

Poser $i \leftarrow i + 1$

fin tant que:

$a_{ij} \neq 0$ ou $i > m$

$a_{ij} \neq 0$ Si $i \leq m$ alors

Echanger lignes i et l

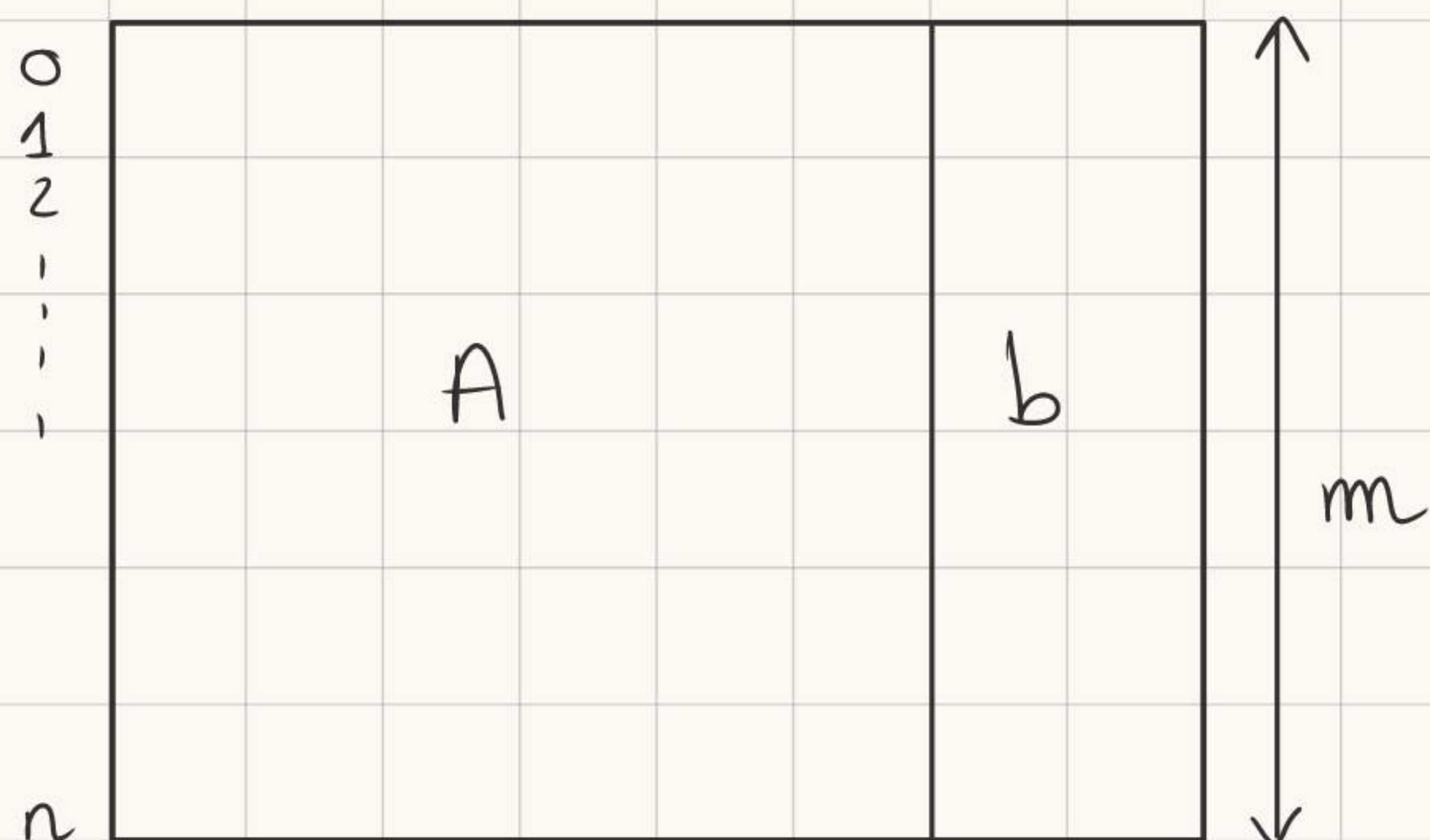
Pivoter sur la ligne l

Poser $l \leftarrow l + 1$

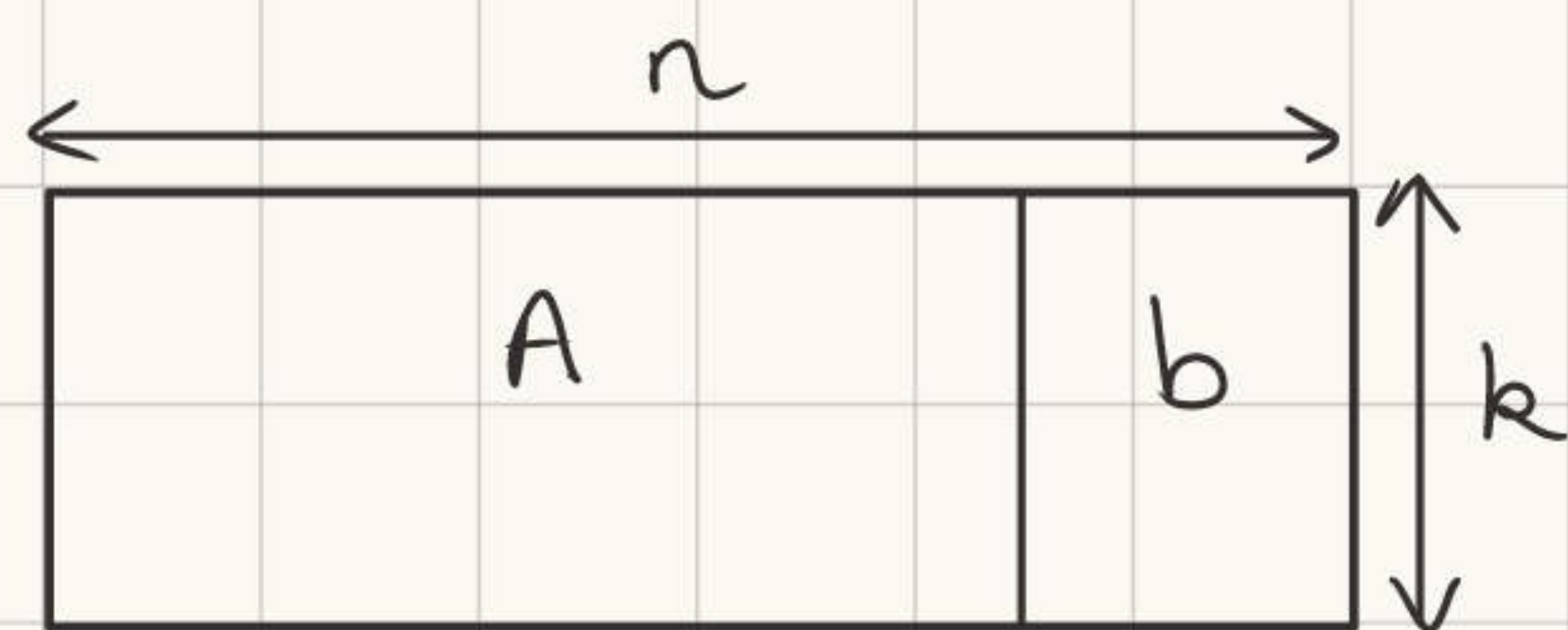
fin Si

fin Pour

$n = \#$ de lignes

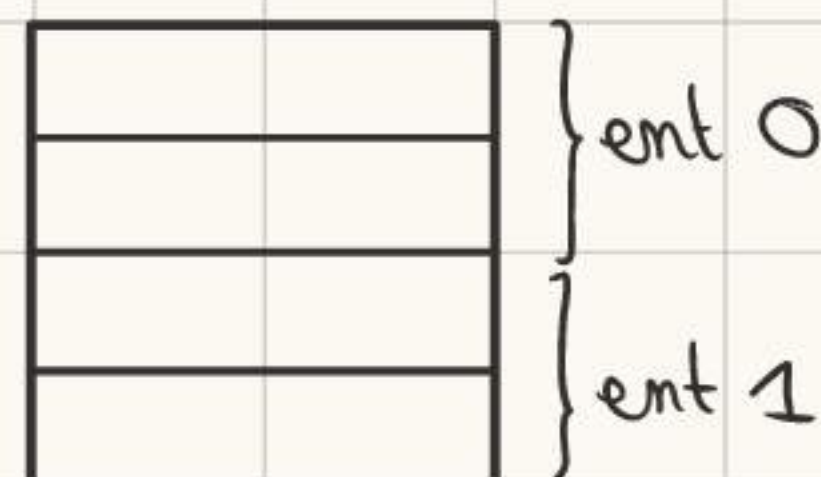


$\left. \begin{array}{c} \\ \\ \\ \vdots \\ \end{array} \right\} m/n$



$m = k \times \# \text{ ent de calc}$

↳ par ex: 2 ent de calc
et 4 lignes :



- * chq ent a k lignes
- * l'ent de calc qui a le pivot envoie sa ligne
- * de chq ent de calc reçoit d'ab la ligne du pivot

pivot = add/sout

avec autres lignes

c'est ça qui se fera en distribué

des msg vont s'éch à cause de ça, $\hat{m}$ si $x=l$

des msg vont aussi s'éch à cause de ça $\hat{m}$ si $\oplus$ easy

Diviser la ligne l par ---

Pour i allant de 1 à m

Soustraire α fois la ligne l (du pivot) à la ligne i

fin pour

① Ecrire le pseudo-code
 * de l'échq de ligne
 * du pivot

② Ecrire le pseudo code distribué

$$k = \lceil nl / n \rceil \text{ lignes de } A.$$

ent calc
 c'est elle qui begin le processus

int $M[k][nc]$;
 int $l = 1$; // n° de ligne du pivot
 int $i = id$;
 int $n = \# \text{ entités}$;

$k \times \text{num ent qui a pivot} = l - 1.$

si l'ent de calc i possède le num de ligne pivot

Si $i = (l - 1) / k$ alors

Poser $m \leftarrow l$

on va parcourir que les lignes à part l
 la 1^{ère} ligne de notre ent

Tant que $m \leq k(i + 1)$

on parcourt l'ent des lignes de l'ent acti

min ligne de notre ent
 $M[m - ki - 1][j] = 0$ faire

Poser $m \leftarrow m + 1$

ent i wrap des lignes entre $(ki) + 1$ et $k(i + 1)$

fin Tant que

Si $m \leq k(i + 1)$ donc pas de ligne de l'ent acti i ms $\neq 0$
 → l'ent i fait l'échq avec elle $\hat{m}$
 car c'est elle qui possède l

de on fait l'échq; l'ent fait l'échq avec elle $\hat{m}$

Env msg 0 à $i + 1$

msg j'ai la ligne

Sinon → on est plus de lignes de l'ent i actuelle

Env msg 1 à $i + 1$

Rec ligne de l et la met de ligne l

fin Si

la ligne du pivot pas de l'ent acti i ms de 1 des entités d'avant

Sinon Si $i > (l - 1) / k$

Rec msg 0/1 de $i - 1$

Si 0 alors pivot déjà found

Si $i < n - 1$ alors

Env 0 à $i + 1$

transmet à l'ent suivante que pivot found
 → si 3

fin Si

Sinon balise 1 si pivot not found yet

Poser $m \leftarrow ki + 1$ → on est l'ent i et on comm à notre 1^{ère} ligne

Tant que $m < k(i + 1)$ et $M[m - ki - 1][j] = 0$ faire:
 pas de lignes de l'ent acti ms 0

Poser $m \leftarrow m + 1$

fin Tant que

$m > k+1$ ou $\neq 0 \rightarrow$ plus de lignes de l'ent courante
 Si $m < k+1$ $\neq 0$ de pivot bound $\rightarrow$ coco l c'est coco l'ent qui cont l

Env la ligne m à $(l-1)/k$

Env 0 à $i+1$ (si $i+1$ existe)

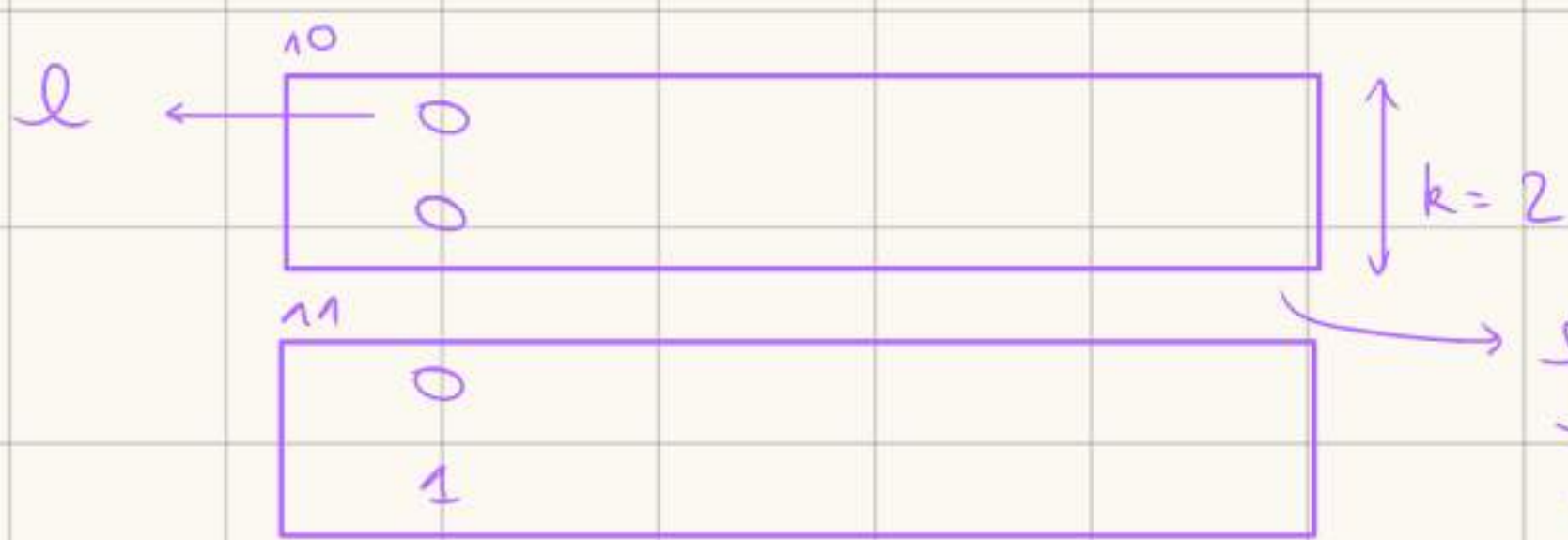
Sinon

plus de lignes de l'ent courante

Env 1 à $i+1$ (si $i+1$ existe)

Rec l de $(l-1)/k$

et si $i+1$ existe pas envoyer "pas de ligne" à $(l-1)/k$



toutes les ent de calc qui ont $i > l$ vont devoir faire des calculs et des échq

l'ent 10 va valoir à l'ent 11 qu'il (10) n'a pas livré de val $\neq 0$ donc 11 doit le chercher $\Rightarrow i=11$

si on a la ligne m
 $(m-1)/k = \text{id de l'ent de calc qui possède la ligne } m$

if $((l-1)/k == i)$

if $((l-1)/k < i)$

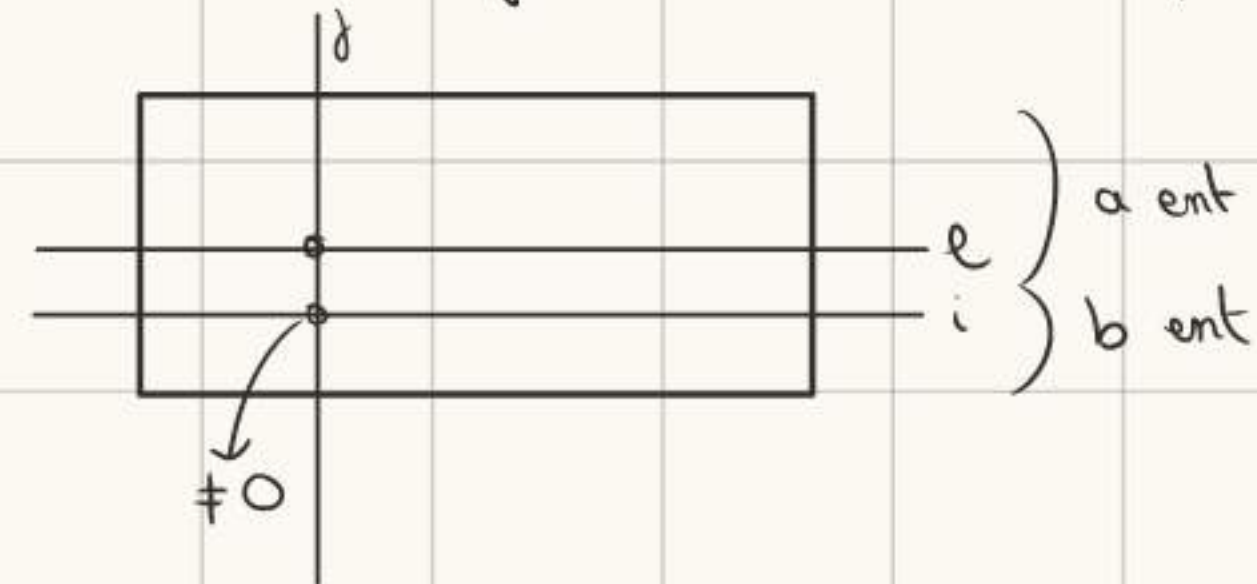
si 0 vals rien à fr du while il ft juste send 0 à $i+1$

si 1 ft cont le while de ft fr la suite

$\hookrightarrow$ si rec 1 vals l'ent de calc d'avant n'avait que des 0

num l atm coco

mais ligne l connue que par 1 entité de calcul



int $M[k][nc]$ $\rightarrow$ tabl d'ent $\rightarrow$ # nb de lignes dont s'occupe 1 entité

messages:

* 0 / 1 (pour le while) } balise 0

* Env de la ligne

* Pas de ligne

* Pas de pivot

$\uparrow$

env de la ligne pour échq = balise 0

env de ligne du pivot de pivot = balise 1

Variables:

l : ligne du pivot

i : identifiant

n : # entités

M : tableau de $k \times (nc + 1)$ entiers où $k = \frac{nl}{n}$

nl/nc : # lignes / colonnes de A.

toutes les entités ont la m[^] val de l

Matrice best : tirer au hasard

$\rightarrow$ mbr de droite (b)
 $\left\{ \begin{array}{l} k \text{ (entre 1 et 10)} \\ nc \text{ (entre 1 et 100)} \\ \text{coeff de } A_b \text{ (entre -1 et 1)} \end{array} \right.$

Communications collectives

MPI - COMM - WORLD

if ($i = (l-1)/k$)

{

for ($j = 0 ; j < n ; j++$)

if ($i \neq j$)

MPI - Send ($M + v$, nc , MPI.DOUBLE, j , 4, MPI - COMM - WORLD);

} else {

MPI - Status status;

MPI - Recv (ligne, nc , MPI.DOUBLE, $(l-1)/k$, 4, MPI - CW, & status);

donc tlm
a la ligne
du pivot

tableau

col y (ça comprend le mbr de D)

buffer dont

l'adresse est $M+v$

il ft avoir la ligne
pour add / soustraire
de matrice

contenu de
la ligne
pas juste num

tous les ent
de calc n'en
sortent que tms au
m moment

⇒ syst synchro

Les lignes de commandes de communic° collective

toutes ces
commandes
synchro le
syst

Diffus°:

→ une ent poss une info (la ligne) et elle la diffuse à ttes les autres

en ayant pu une cast ou flood → ms no need ici pq on sait que tlm est lié

elle s'exé
sur ttes ent
en m tps
de m 1 ent
calc a pas fin
son calc, les
autres wait

recopier $M+v$ dans ligne sur $(l-1)/k$:

MPI - Bcast (ligne, nc , MPI.DOUBLE, $(l-1)/k$, MPI - COMM - WORLD);

buffer

1^{er} fois qu'on ft broadcast à pt certⁿ ent

de calcul, MPI ft flood echo: cont un π puis les broade qu'en fait après,
depuis cette ent de calc à racine,
il ft cast de cet arbre.

Concentration:

le buffer
ligne de l'ent
de calc source
s'ent de les buff
ligne des autres

#élé env

MPI - Gather (buffer - e, $k-e$, MPI - TYPE, buffer - r, $k-r$, MPI - TYPE, r , MPI - C - W);

il y a 2 buffers
cette cmd = env pour ttes ent de calc
de ces buffers doit 3 chez tlm

Op(4,5)
pt 4+5
ou 4x5

→ ft préciser l'op°

1 seul pcp le m m env et rec

MAX
MIN
PROD
SUM
LOR
LAND

need to be
associatifs
pq si op° se font
do ordre ≠ need
to donner m res

Réduction:

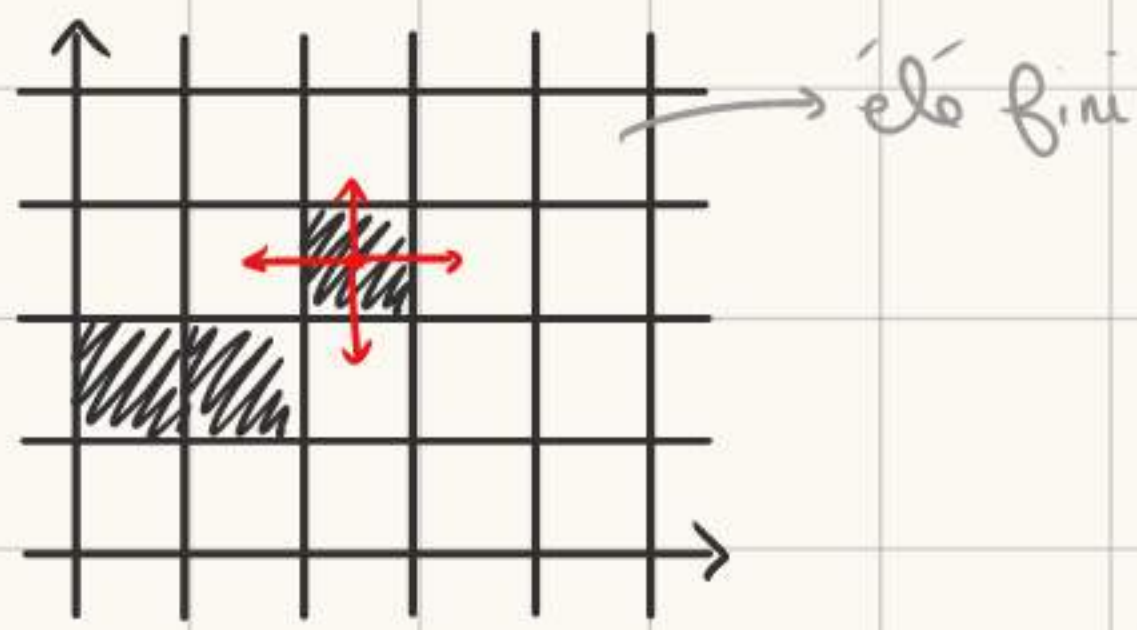
MPI - Reduce (buffer - e, buffer - r, k , MPI - TYPE, MPI - OP, r , MPI - C - W);

→ c concentr°: ttes les ent calc ent val à env à r
ms à chq recept° y a op° pour garder taille info égale
de à la fin taille buffer - e = taille buffer - r

Barrière de synchronisat°

→ synchronise syst dist sans
faire d'env / recep

MPI - Barrier (MPI - COMM - WORLD); → tte ce qu'elle fait c'est



MPI - COMM - WORLD

se termine qd tous les autres ont de calc s'éc.

ép° dérivée partielle résolue par méthd d'Euler
(domN géo: dim, esp euclidien)

on a un espace de
lept en vt coco val de
chq pt de sol°

on ga coupe domN
et do ces élé finis
on supp que q^T qu'on vt
estimée est cote.

Partit°

en SD, chq ent s'occupe
d'un élé et communique
avec ses 2 d voisins
=> need communicateurs
qui permettent d'accéder aux voisins
(un comm en ligne
un en colonne etc)

et on va res syst en
regardt ce qu'il se passe
do élé voisin

MPI - Comm - split (MPI - COMM - WORLD comm, couleur, n - id, & nouveau - comm);

MPI - Comm nouveau - comm

couleur → fonction d'identifiant
n - id → (id - id % 2) / 2 → new id

On prend 1 comm et ga
soit comm coupé en 2
qu'on ft une vls broadcast etc

Ce qui donne par ex :

id	0	1	2	3	4
n - id	0	0	1	1	2

same id
ms do diff parties du split

pair impair

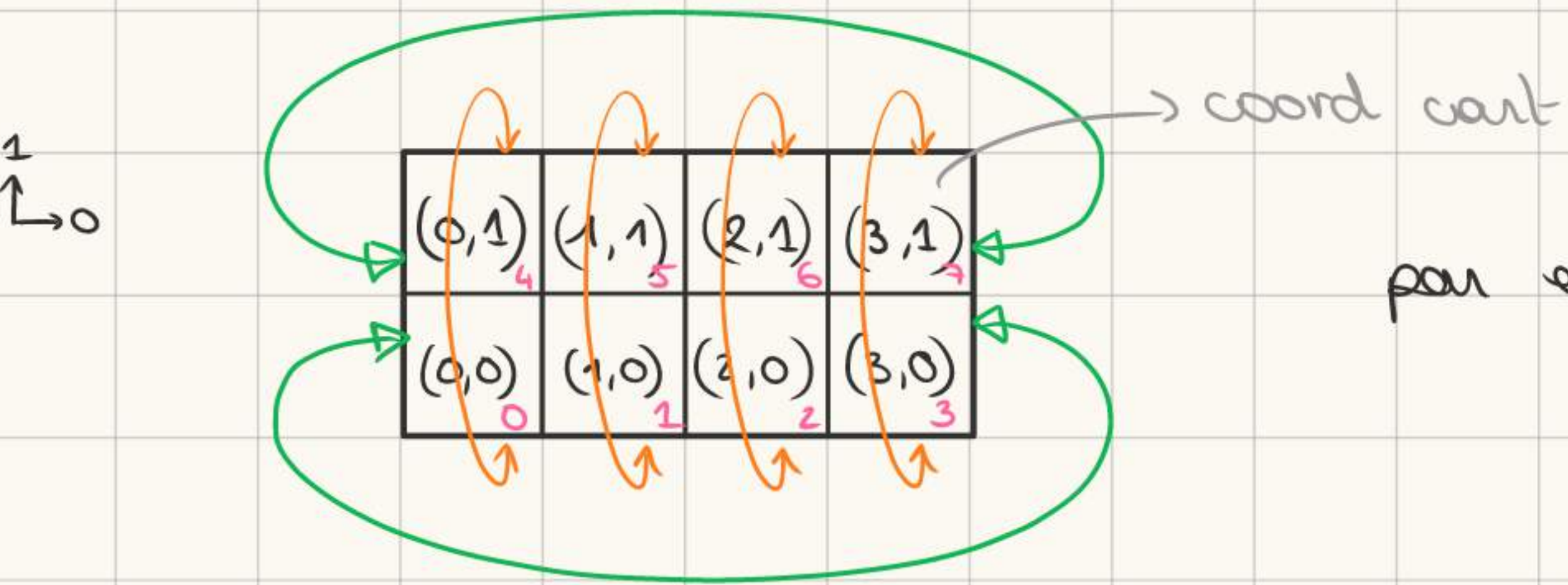
ga fait en ligne?

MPI - Bcast (b, 1, MPI - INT, 0, nouveau - comm);

en utilisant ce communicateur,
Donc pour passer d'une entité d'identifiant
d'une partie à l'autre il ft repasser
par MPI - COMM - WORLD

Topologie cartésienne :

chq ent communique avec ses voisins selon
chq dimension.



par ex, 6 est voisin direct de 5, 7 et 2

MPI - Cart - create (comm, d, tailles, périodes, a, & nouveau - comm);

Recherche de voisins;

MPI - Cart - shift (comm, k, s, & precv, & suiv);

topologie
cartésienne

distance : 1, 2, 3

selon laq^l on fait
recherche (ici 0 ou 1)

reorder
0 ou 1 si autorise
MPI à réorg les
identifiants

on est en topo cart
de si on va do une
direct° et qu'on reg
à G et à D à dist 2
alo 2 voisins => & prec
& suiv

1 2 3 4 et prec = PROC - NULL

MPI-Proc-NULL;

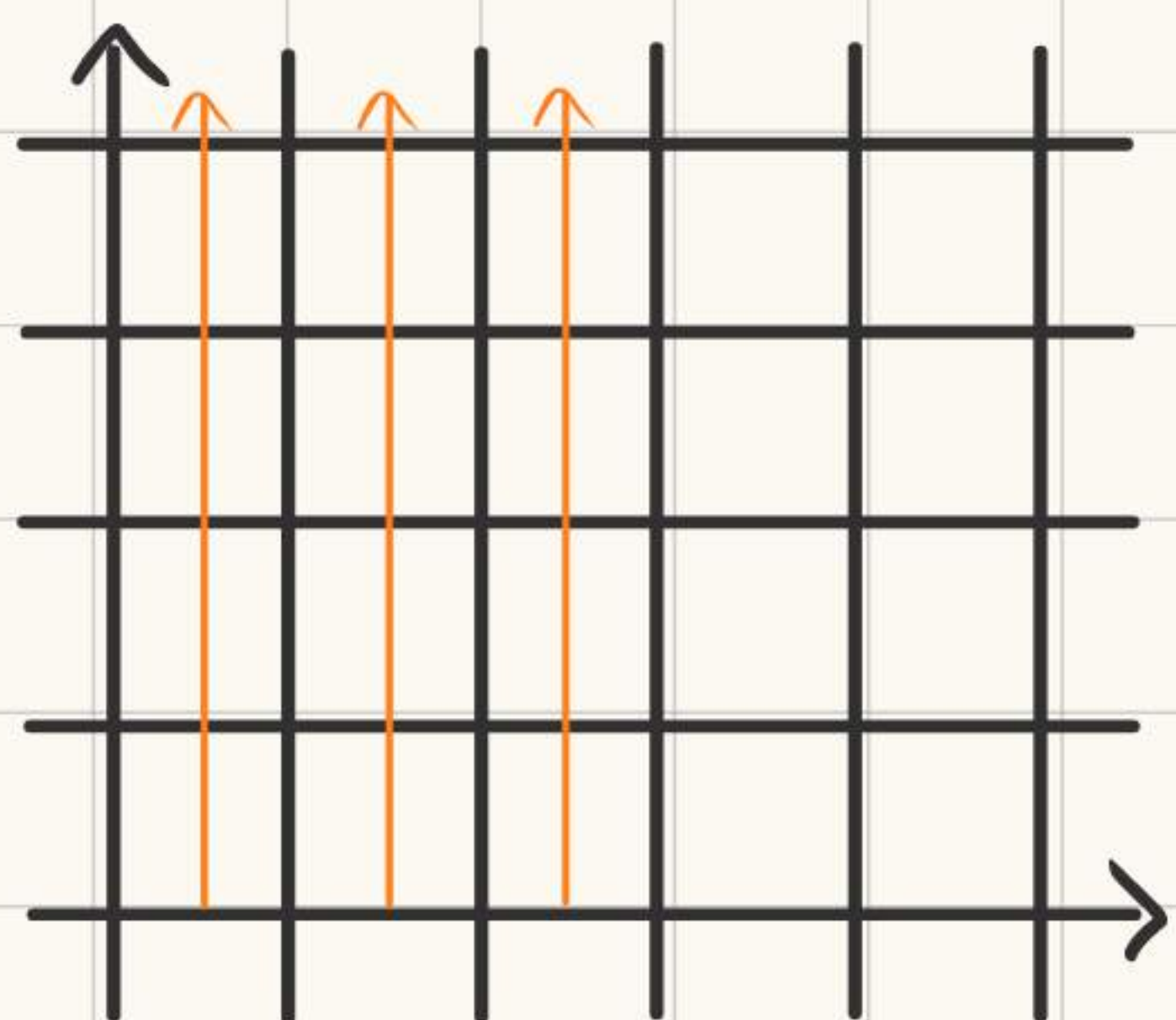
Recherche de coordonnées :

MPI-Cart-coords (comm, id, d, coord)

Annotations:

- comm: topo cont
- id: le nôtre
- d: dim
- coord: adresse du tableau des coord qui on va recevoir
- coord: [coord1, coord2, ..., coordd]
- coord: pas coord au celle qui on reçoit?

Sous communications dimensionnelles



MPI-Cart-sub (comm, d - restantes, & nouveau-comm);

Annotations:

- d: qd dir° on garde dans comm.
- reste: [0, 1, ..., 0, 1]
- d: si je veux comm en f° des lignes, [1, 0] est [0, 1]
- reste: pas de nvcomm si c'est ds.

par broadcast en ligne ou en col par ex

ordre considéré & préservé si 2 env par 1 im proc.